

Software Analysis

Dataflow Analysis
(Part I)

Gordon Fraser
Lehrstuhl für Software Engineering II

Example

`x := 8;`

`y := x;`

`z := 0;`

Will it ever terminate?

`while y > -1 do`

`x := x / y;` Can there be a division by zero?

`y := y - 2;`

`z := 5;`

Is this value ever used?

Outline

1. Reaching Definitions
2. Reachable Uses
3. DU Chains, UD Chains
4. Iterative Dataflow Analysis

Data Dependence

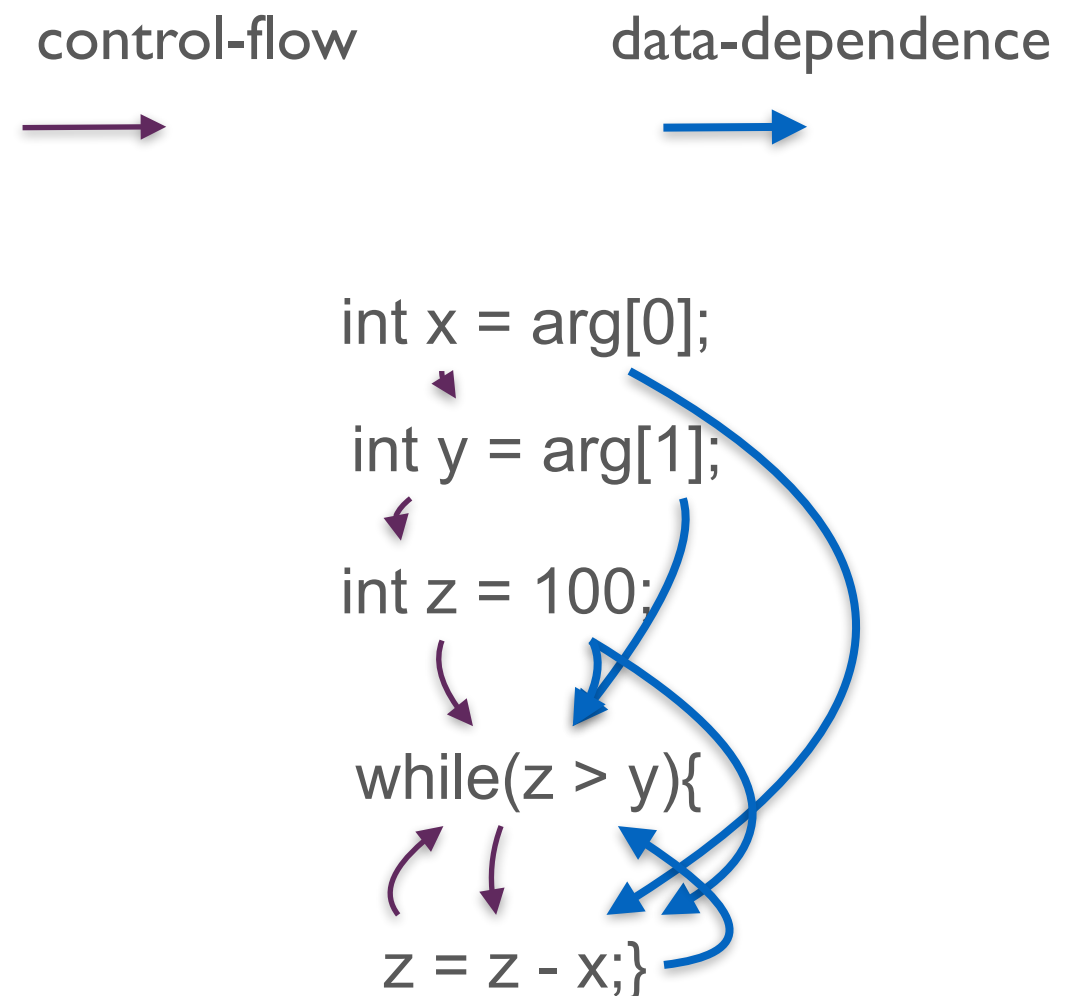
Follow the **flow of data** through a program.

Statement **b** is data-dependent upon statement **a** if:

A variable **v** is read at statement **b**, which was written at statement **a**.

There is a path from **a** to **b** in the control flow graph that does not contain any other statements that write to **v**.

I.e., there is a Def-Use pair



Data Dependence Example

```
public int gcd(int x, int y) {  
    int tmp;  
  
    while(y != 0) {  
        tmp = x % y;  
        x = y;  
        y = tmp;  
    }  
  
    return x;  
}
```

Data Dependence Example

```
public int gcd(int x, int y) {  
    int tmp;  
  
    while(y != 0) {  
        tmp = x % y;  
        x = y;  
        y = tmp;  
    }  
  
    return x;  
}
```

Definitions

Data Dependence Example

```
public int gcd(int x, int y) {  
    int tmp;  
  
    while(y != 0) {  
        tmp = x % y;  
        x = y;  
        y = tmp;  
    }  
  
    return x;  
}
```

Uses

Def-Use Pairs

```
public int gcd(int x, int y) {  
    int tmp;
```

```
    while(y != 0) {  
        tmp = x % y;  
        x = y;  
        y = tmp;  
    }
```

```
    return x;  
}
```

Def-Use Pairs

Def-Use Pairs

```
public int gcd(int x, int y) {  
    int tmp;  
  
    while(y != 0) {  
        tmp = x % y;  
        x = y;  
        y = tmp;  
    }  
  
    return x;  
}
```

Def-Use Pairs

Def-Use Pairs

- A *def-use (du) pair* represents a data dependence between a location where a value is produced and the location where it is used.
- *Definition*: Variable is assigned a value
 - Declarations
 - Initialisation
 - Assignment
 - Parameters of function calls
- *Use*: Value of variable is read
 - Expressions
 - Conditional statements
 - Use as parameter of a function call
 - Return statements

Def-Use Pairs

- A *definition-clear path* is a path in the CFG from a definition to a use of the same variable, such that the variable is not re-defined along the path
 - If there is no other definition that overwrites (*kills*) the previous definition
- A Def-Use Pair requires a definition-clear path from the def to the use

Def-Use Pairs

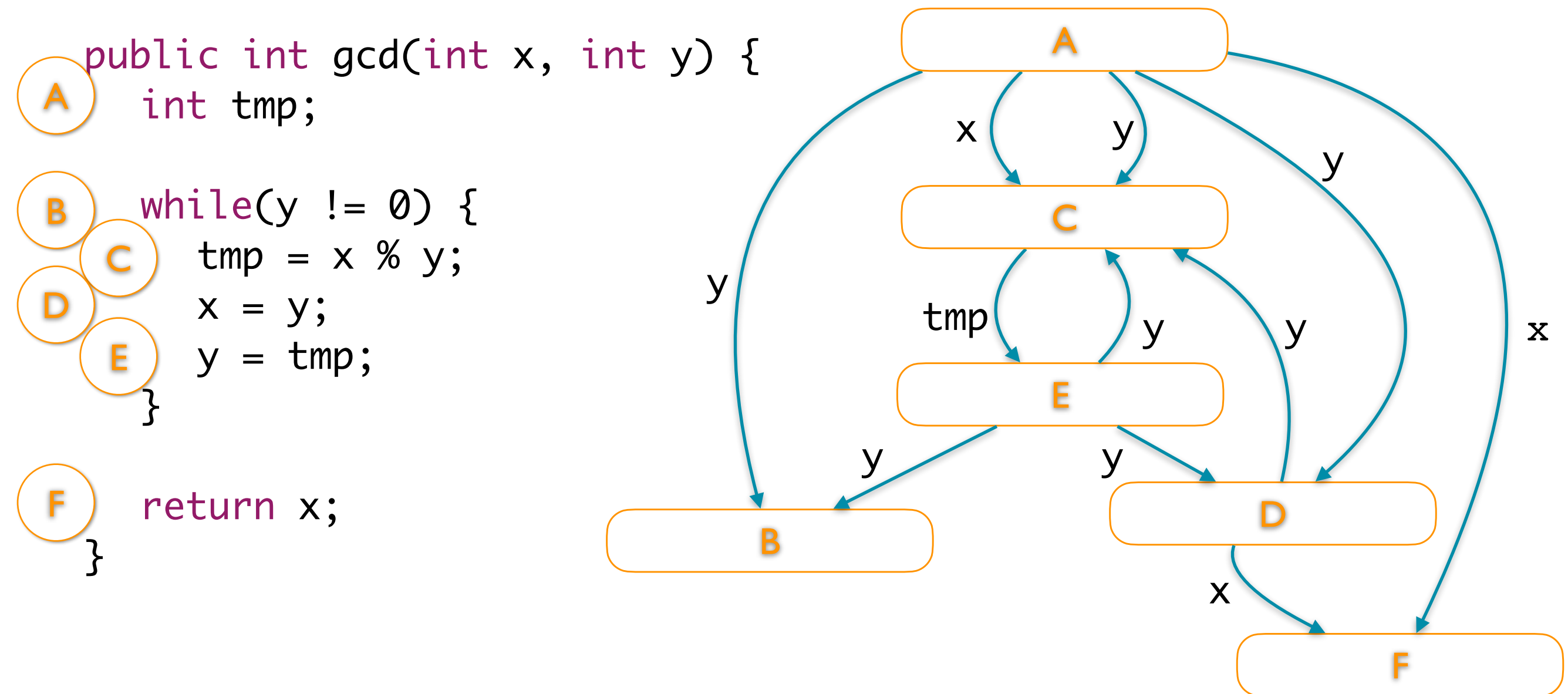
```
public int gcd(int x, int y) {  
    int tmp;  
  
    while(y != 0) {  
        tmp = x % y;  
        x = y;  
        y = tmp;  
    }  
  
    return x;  
}
```

Def-Use Pair vs Def-Use Path

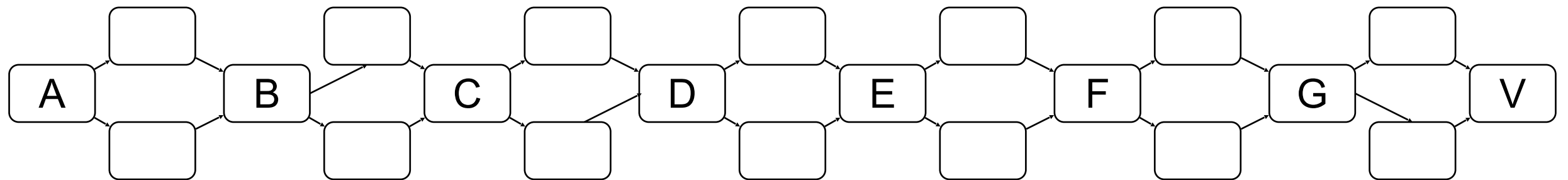
- A def-use pair can have more than one def-clear path
- Definition Clear Path:
Path from def to use without killing def
- Definition-Use Pair (DU Pair)
Definition+Use with at least 1 def-clear path

Data Dependence Graph

- Nodes: as in the control flow graph (CFG)
- Edges: def-use pairs, labelled with the variable name



Exponential paths (even without loops)



2 paths from A to B

4 from A to C

8 from A to D

16 from A to E

...

128 paths from A to V

*Tracing each path is not
efficient, and we can do
much better.*

Recap: Computing Dominators

Result: A map of dependence relations D

```
dominance( $CFG$ ) begin
   $entry \leftarrow entryNode(CFG);$ 
   $D(entry) \leftarrow entry;$ 
  for  $n \in CFG.nodes - \{entry\}$  do
     $D(n) \leftarrow CFG.nodes;$ 
  end
   $changed \leftarrow true;$ 
  while  $changed$  do
     $changed \leftarrow false;$ 
    for  $n \in CFG.nodes - \{entry\}$  do
       $currentDominators \leftarrow D(n);$ 
       $newDominators \leftarrow n \cup (\bigcap D(p))$  for all immediate predecessors  $p$  of  $n$ ;
      if  $currentDominators \neq newDominators$  then
         $D(n) \leftarrow newDominators;$ 
         $changed \leftarrow true;$ 
      end
    end
  end
  return  $D$ 
end
```


Recap: Computing Dominators

Basic Dataflow Equations:

$$\text{Dom}(n_e) = \{n_e\}$$

$$\text{Dom}(n) = \left(\bigcap_{s \in \text{pred}(n)} \text{Dom}(s) \right) \cup \{n\}$$

- Initialise $\text{Dom}(n)$ with set of all nodes for all n except entry node
- while some $\text{Dom}(n)$ changed
 - recalculate $\text{Dom}(n)$ for all n with above formula

Equations of node E ($y = \text{tmp}$)

Calculate **reaching definitions** at E
can be done in terms of its
immediate predecessor D

```
public class GCD {  
    public int gcd(int x, int y) {  
        int tmp;           // A: def x, y, tmp  
        while (y != 0) {    // B: use y  
            tmp = x % y;     // C: def tmp; use x, y  
            x = y;           // D: def x; use y  
            y = tmp;        // E: def y; use tmp  
        }  
        return x;           // F: use x  
    }  
}
```

$$\text{ReachIn}(E) = \text{ReachOut}(D)$$

$$\text{ReachOut}(E) = (\text{ReachIn}(E) \setminus \{y_A\}) \cup \{y_E\}$$

Equations of node B (while (y != 0))

*This line has two predecessors:
before the loop (A) and **end** of the
loop (E).*

```
public class GCD {  
    public int gcd(int x, int y) {  
        int tmp;           // A: def x, y, tmp  
        while (y != 0) {    // B: use y  
            tmp = x % y;     // C: def tmp; use x, y  
            x = y;           // D: def x; use y  
            y = tmp;         // E: def y; use tmp  
        }  
        return x;          // F: use x  
    }  
}
```

$$\text{ReachIn}(B) = \text{ReachOut}(A) \cup \text{ReachOut}(E)$$

$$\text{ReachOut}(A) = \{x_A, y_A, \text{tmp}_A\} \quad // \text{gen}(A)$$

$$\text{ReachOut}(E) = (\text{ReachIn}(E) \setminus \{y_A\}) \cup \{y_E\}$$

General Equations for Reach Analysis

$$\text{ReachIn}(n) = \bigcup_{m \in \text{pred}(n)} \text{ReachOut}(m)$$

$$\text{ReachOut}(n) = (\text{ReachIn}(n) \setminus \text{kill}(n)) \cup \text{gen}(n)$$

$$\text{gen}(n) = \{ v_n \mid v \text{ is defined or modified at } n \}$$

$$\text{kill}(n) = \{ v_x \mid v \text{ is defined or modified at } x, x \neq n \}$$

Reaching Definitions Algorithmus

```
for all n in nodes
```

```
    ReachIn(n) = {}
```

```
    ReachOut(n) = {}
```

```
worklist = nodes
```

```
while(worklist not empty):
```

```
    n = take node from worklist
```

```
    oldVal = ReachOut(n)
```

```
    Apply flow equations for ReachIn(n) / ReachOut(n)
```

```
    if(ReachOut(n) != oldVal)
```

```
        workList = workList + succ(n)
```

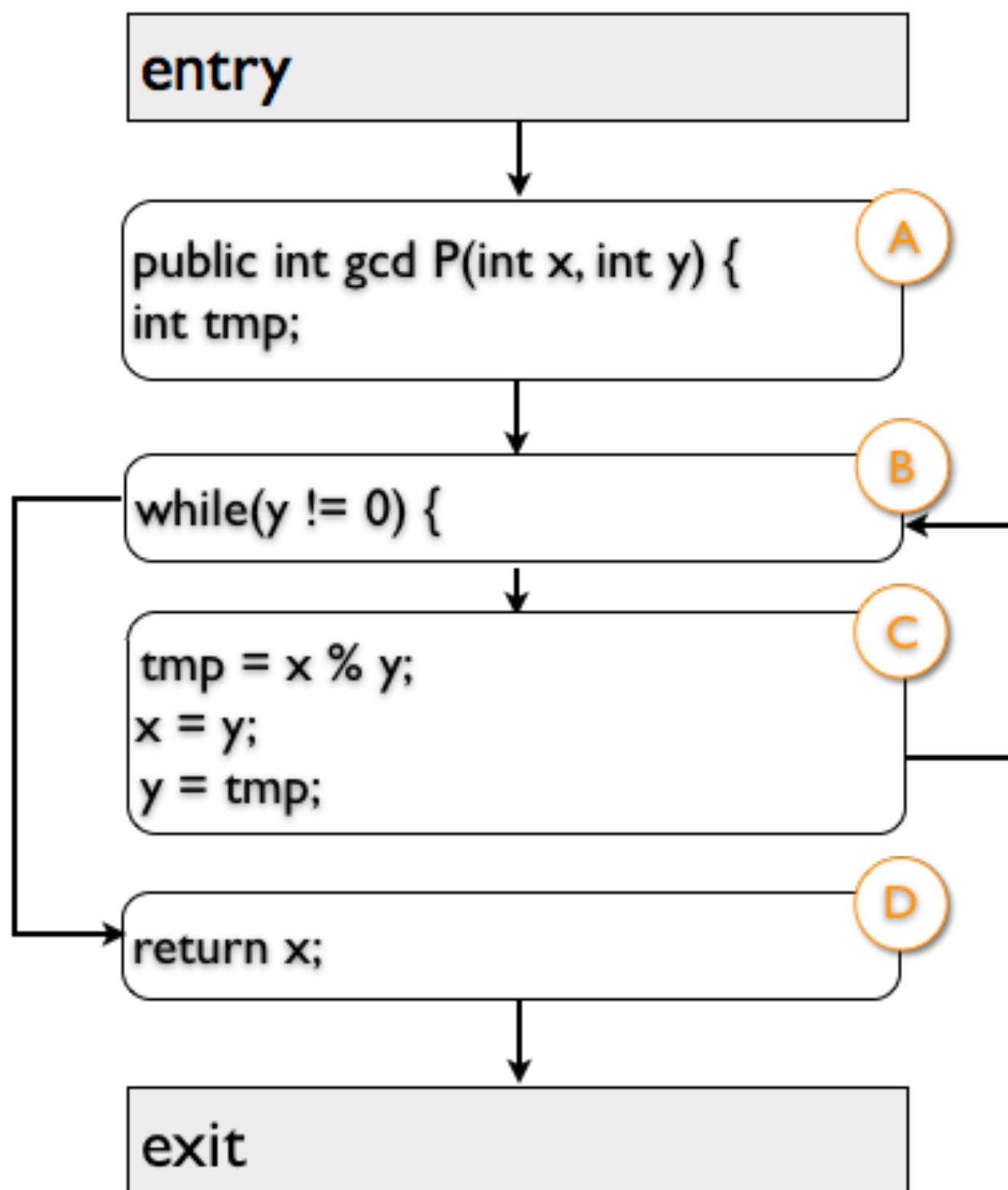
Reaching Definitions Example

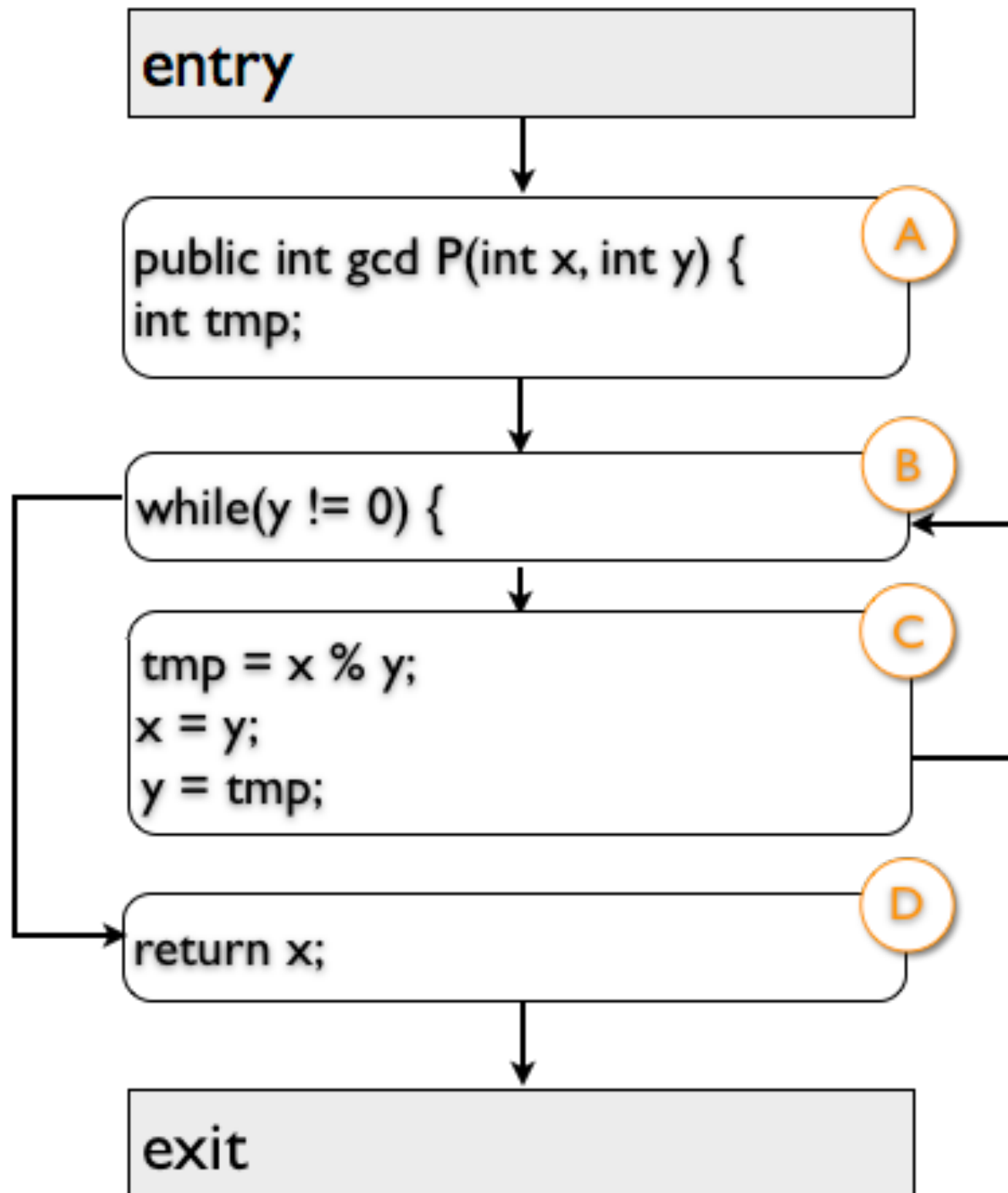
```

A public int gcd(int x, int y) {
    int tmp;

    B while(y != 0) {
        tmp = x % y;
        C x = y;
        y = tmp;
    }

    D return x;
}
```





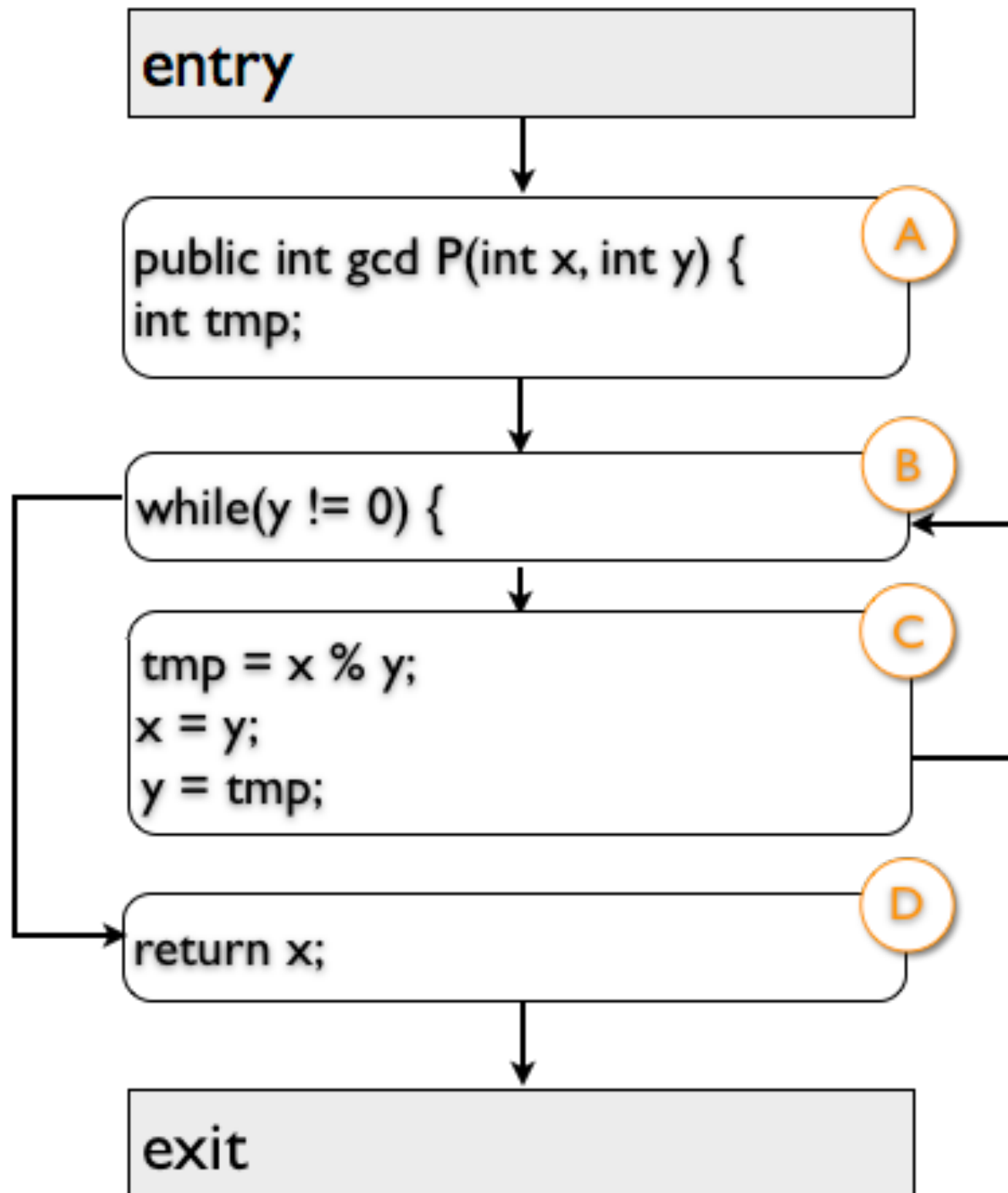
Reach(A) = {}
ReachOut(A) = {}

Reach(B) = {}
ReachOut(B) = {}

Reach(C) = {}
ReachOut(C) = {}

Reach(D) = {}
ReachOut(D) = {}

worklist = {A, B, C, D}



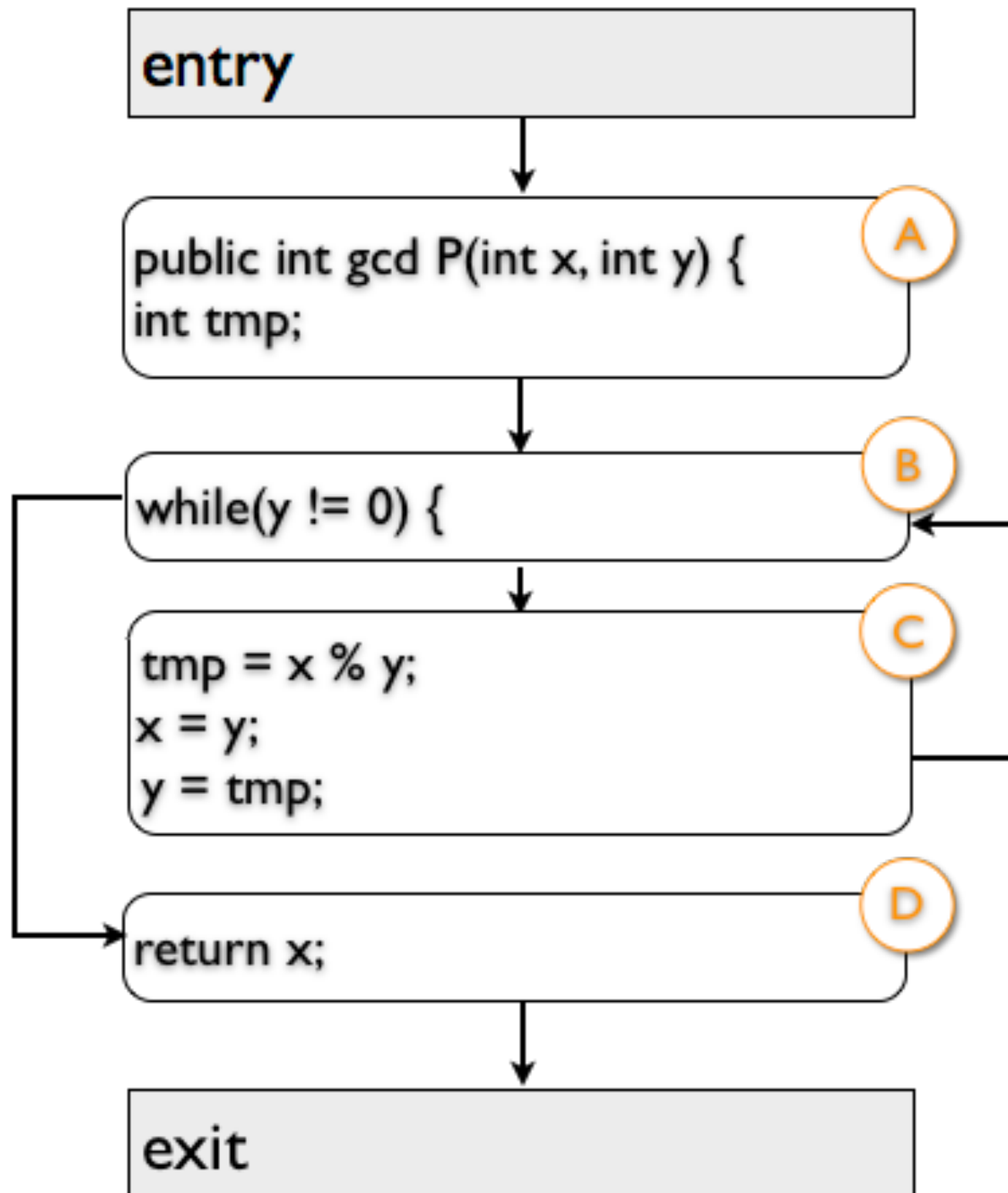
Reach(A) = {}
ReachOut(A) = {}

Reach(B) = {}
ReachOut(B) = {}

Reach(C) = {}
ReachOut(C) = {}

Reach(D) = {}
ReachOut(D) = {}

worklist = {B, C, D}
n = A



Reach(A) = {}

ReachOut(A) = {x_A, y_A, tmp_A}

Reach(B) = {}

ReachOut(B) = {}

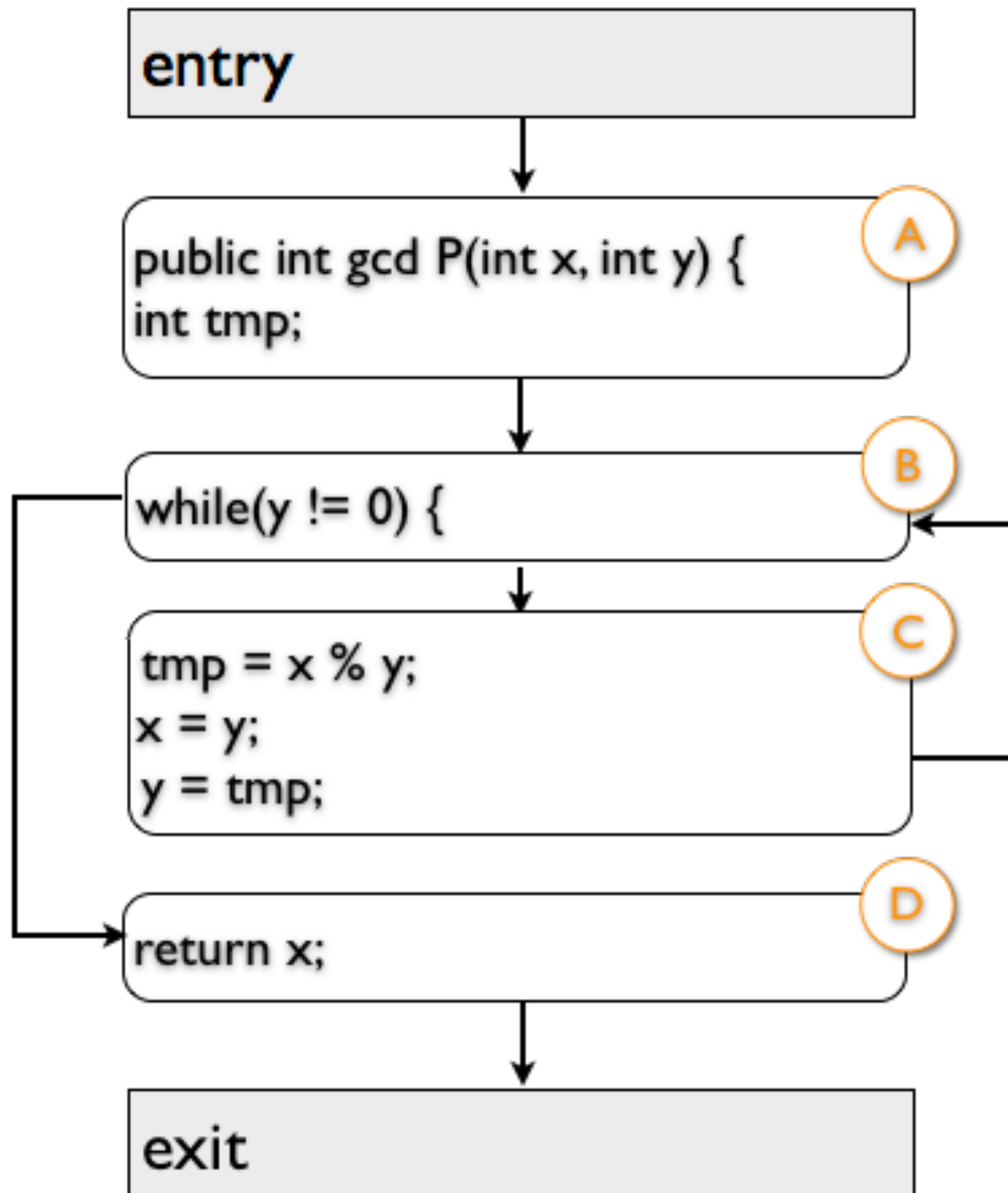
Reach(C) = {}

ReachOut(C) = {}

Reach(D) = {}

ReachOut(D) = {}

worklist = {B, C, D, B}



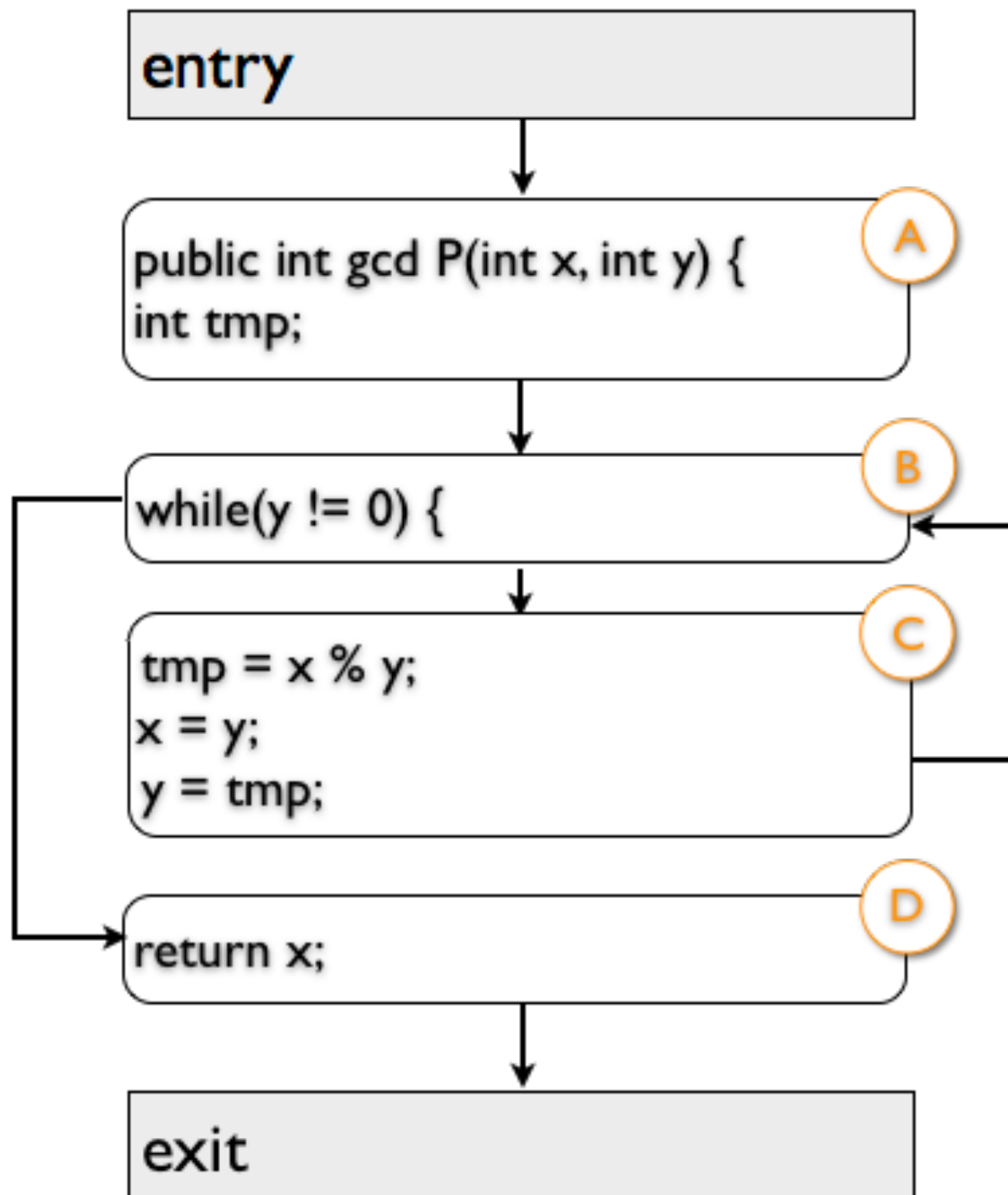
$\text{Reach}(A) = \{\}$
 $\text{ReachOut}(A) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(B) = \{\}$
 $\text{ReachOut}(B) = \{\}$

$\text{Reach}(C) = \{\}$
 $\text{ReachOut}(C) = \{\}$

$\text{Reach}(D) = \{\}$
 $\text{ReachOut}(D) = \{\}$

$\text{worklist} = \{C, D, B\}$
 $n = B$



Reach(A) = {}

ReachOut(A) = {x_A, y_A, tmp_A}

Reach(B) = {x_A, y_A, tmp_A}

ReachOut(B) = {x_A, y_A, tmp_A}

Reach(C) = {}

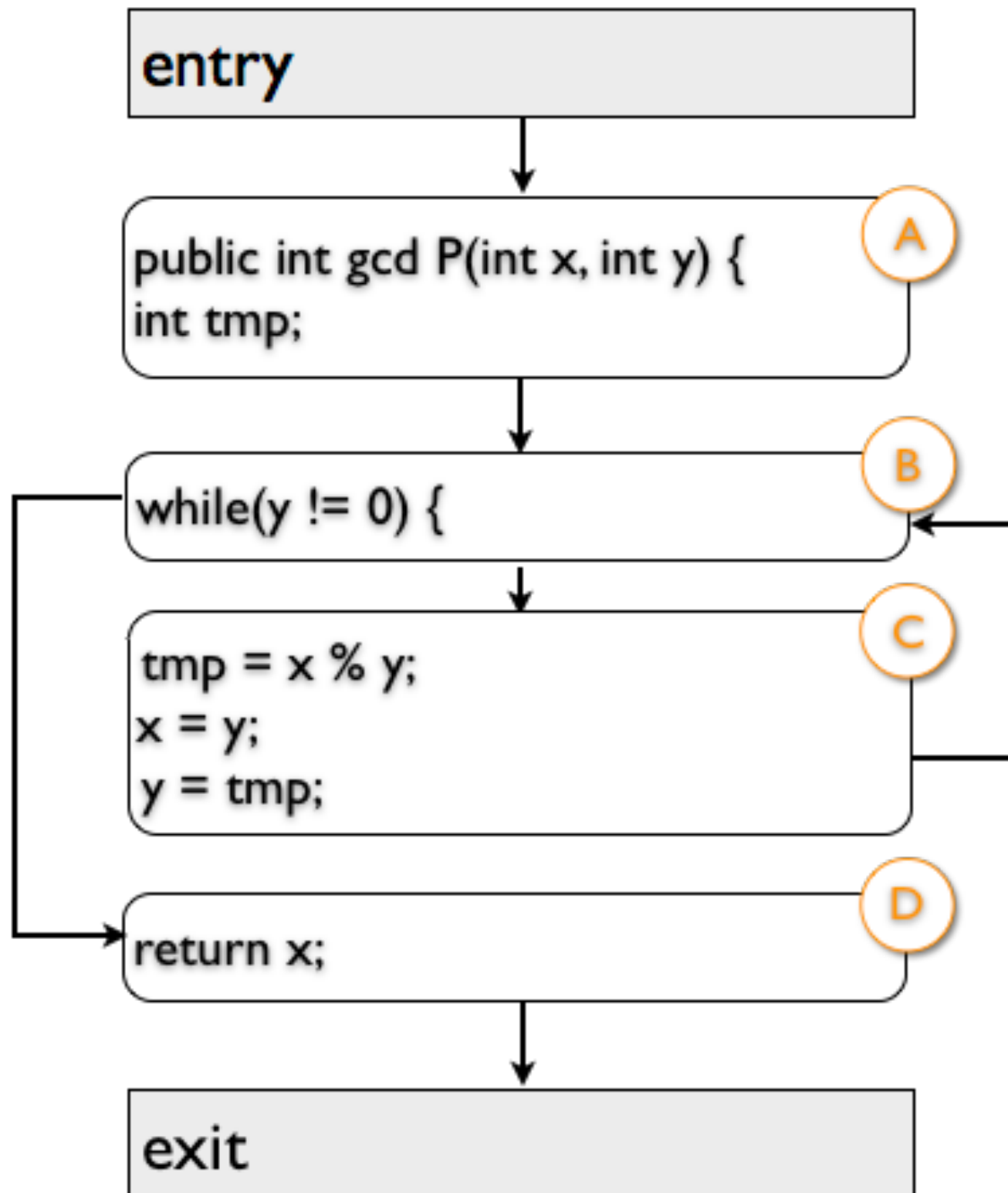
ReachOut(C) = {}

Reach(D) = {}

ReachOut(D) = {}

worklist = {C, D, B}

n = B



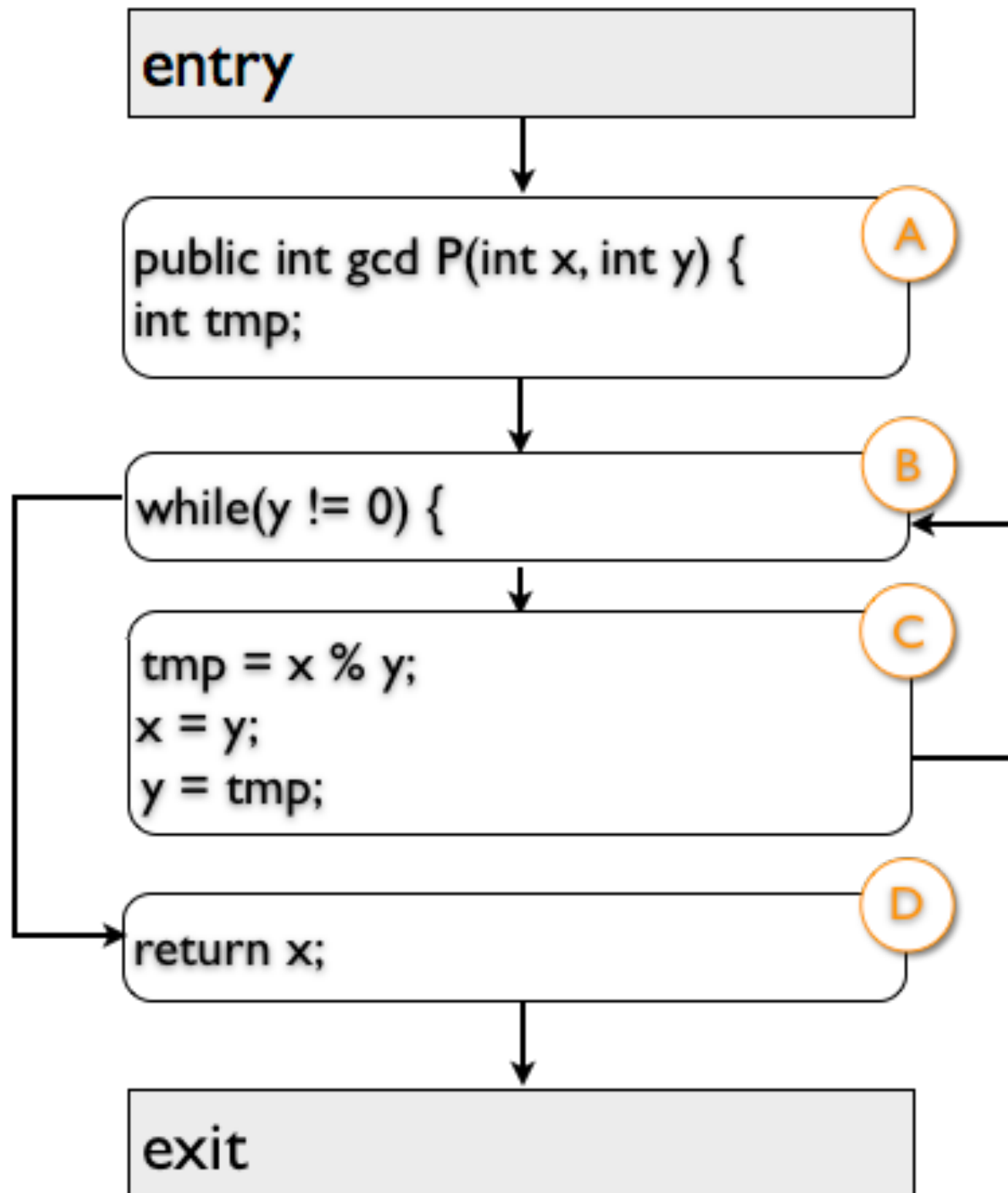
$\text{Reach}(A) = \{\}$
 $\text{ReachOut}(A) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(B) = \{x_A, y_A, \text{tmp}_A\}$
 $\text{ReachOut}(B) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(C) = \{\}$
 $\text{ReachOut}(C) = \{\}$

$\text{Reach}(D) = \{\}$
 $\text{ReachOut}(D) = \{\}$

worklist = {C, D, B, C, D}



$\text{Reach}(A) = \{\}$

$\text{ReachOut}(A) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(B) = \{x_A, y_A, \text{tmp}_A\}$

$\text{ReachOut}(B) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(C) = \{\}$

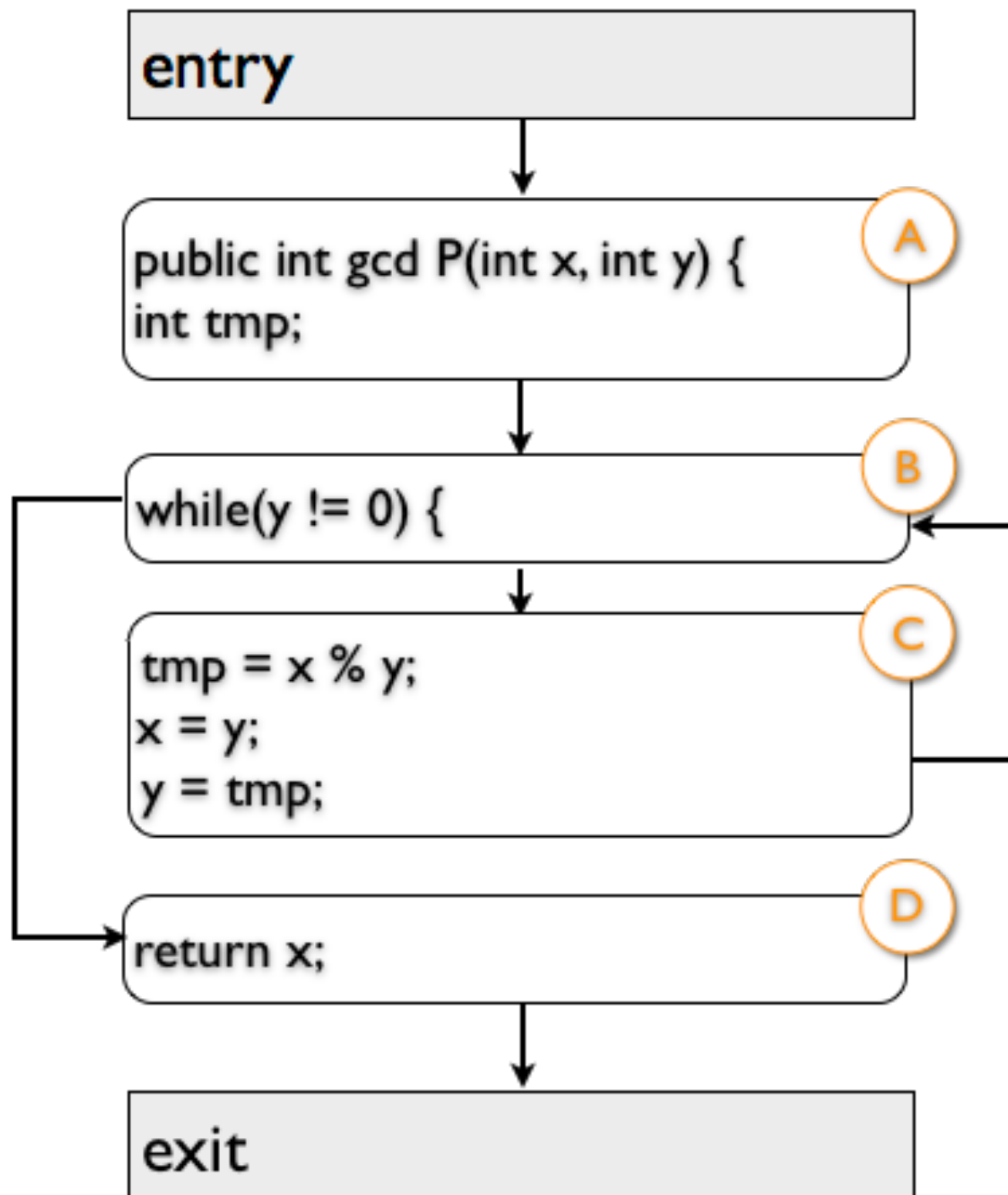
$\text{ReachOut}(C) = \{\}$

$\text{Reach}(D) = \{\}$

$\text{ReachOut}(D) = \{\}$

$\text{worklist} = \{D, B, C, D\}$

$n = C$



Reach(A) = {}

ReachOut(A) = {x_A, y_A, tmp_A}

Reach(B) = {x_A, y_A, tmp_A}

ReachOut(B) = {x_A, y_A, tmp_A}

Reach(C) = {x_A, y_A, tmp_A}

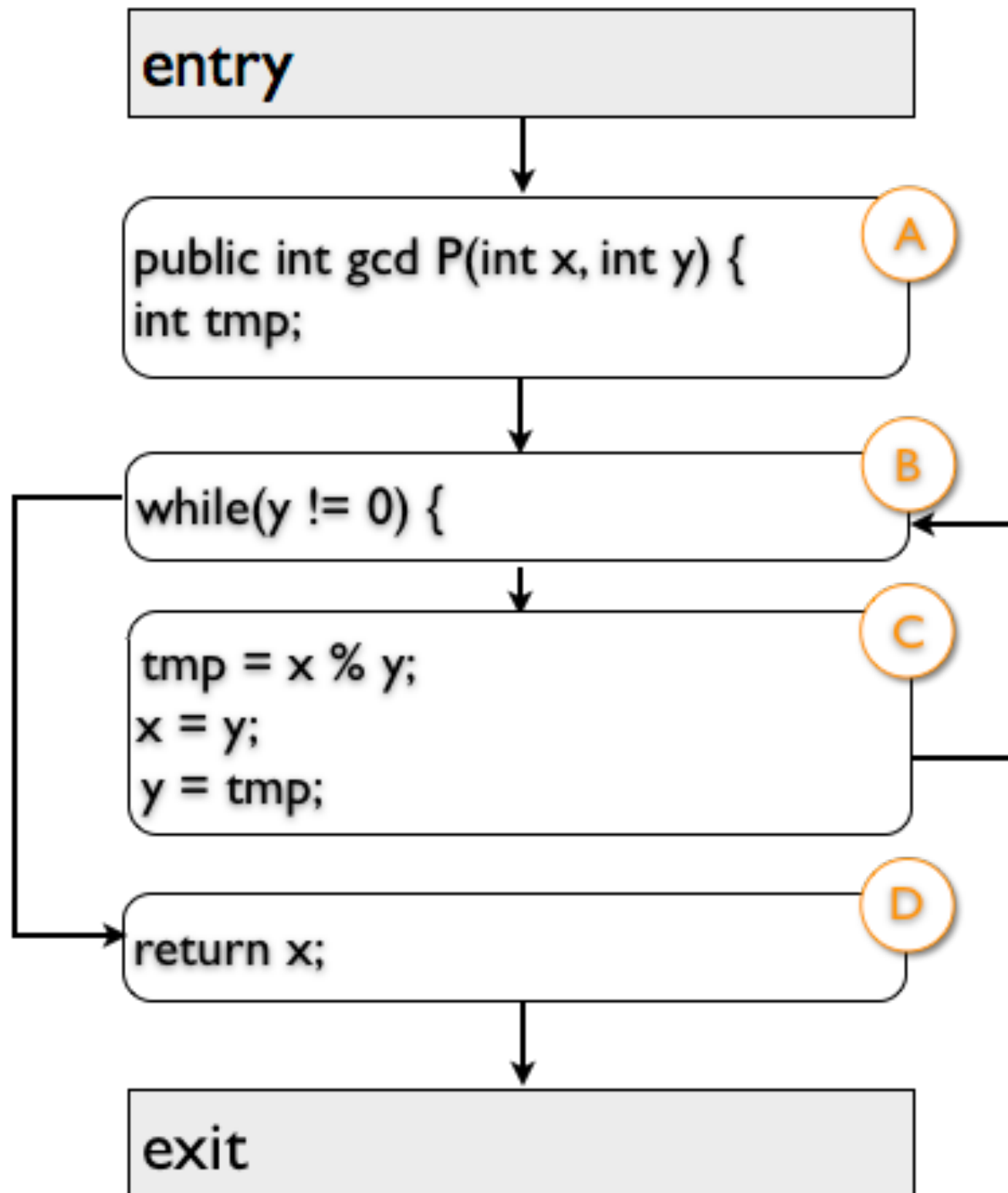
ReachOut(C) = {x_C, y_C, tmp_C}

Reach(D) = {}

ReachOut(D) = {}

worklist = {D, B, C, D}

n = C



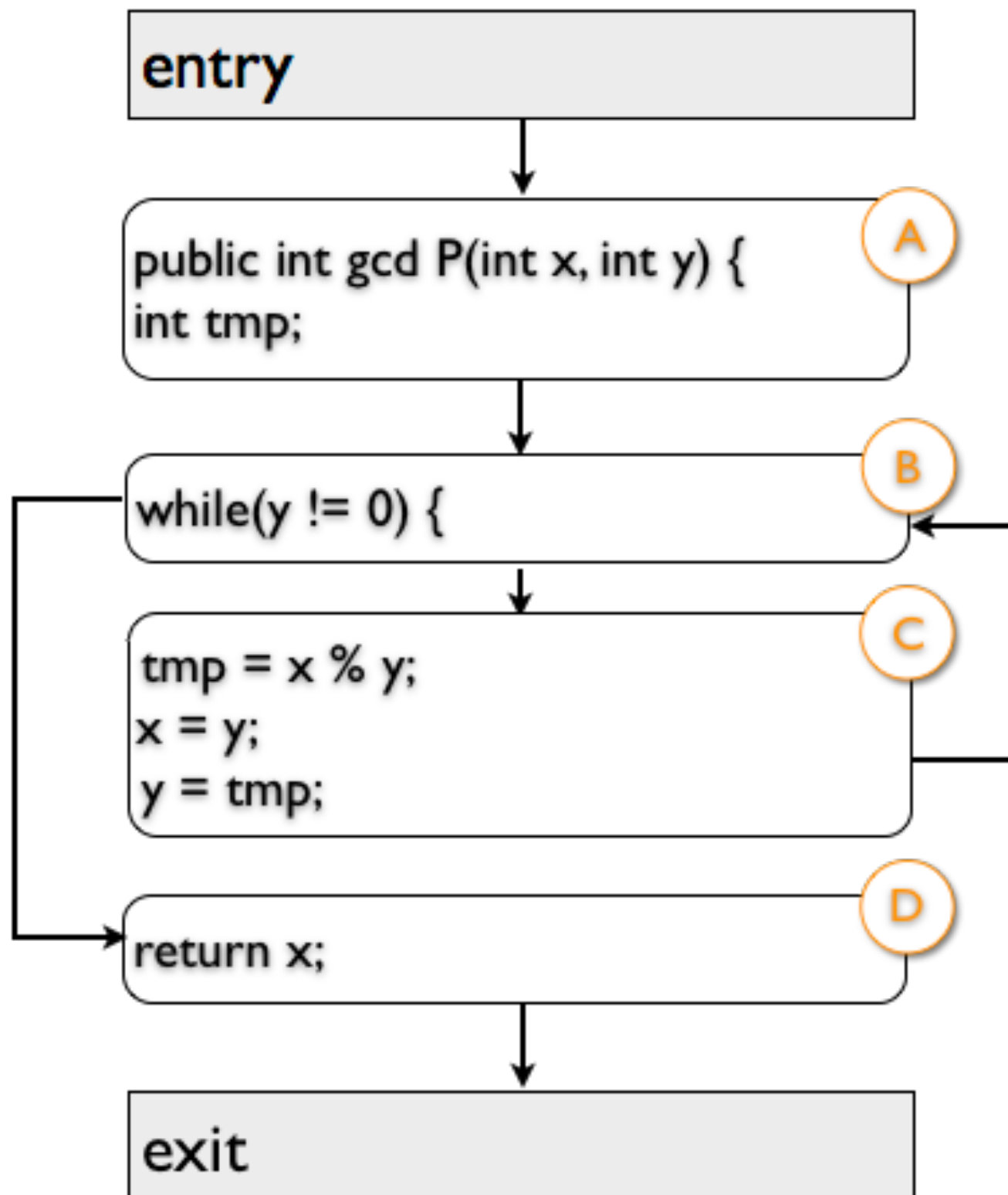
Reach(A) = {}
ReachOut(A) = {x_A, y_A, tmp_A}

Reach(B) = {x_A, y_A, tmp_A}
ReachOut(B) = {x_A, y_A, tmp_A}

Reach(C) = {x_A, y_A, tmp_A}
ReachOut(C) = {x_C, y_C, tmp_C}

Reach(D) = {}
ReachOut(D) = {}

worklist = {D, B, C, D, **B**}



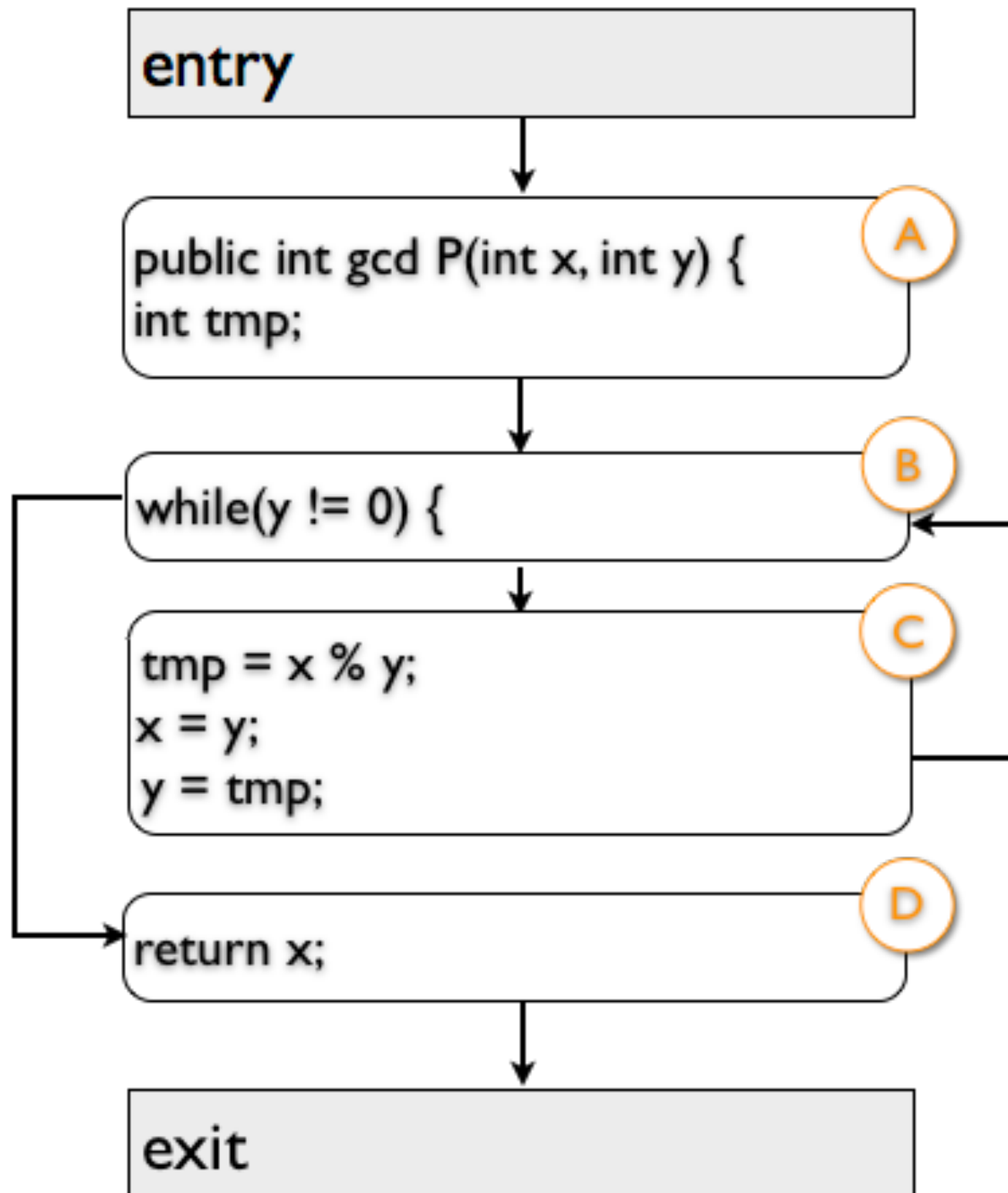
$\text{Reach}(A) = \{\}$
 $\text{ReachOut}(A) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(B) = \{x_A, y_A, \text{tmp}_A\}$
 $\text{ReachOut}(B) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(C) = \{x_A, y_A, \text{tmp}_A\}$
 $\text{ReachOut}(C) = \{x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(D) = \{\}$
 $\text{ReachOut}(D) = \{\}$

$\text{worklist} = \{B, C, D, B\}$
 $n = D$



$\text{Reach}(A) = \{\}$

$\text{ReachOut}(A) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(B) = \{x_A, y_A, \text{tmp}_A\}$

$\text{ReachOut}(B) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(C) = \{x_A, y_A, \text{tmp}_A\}$

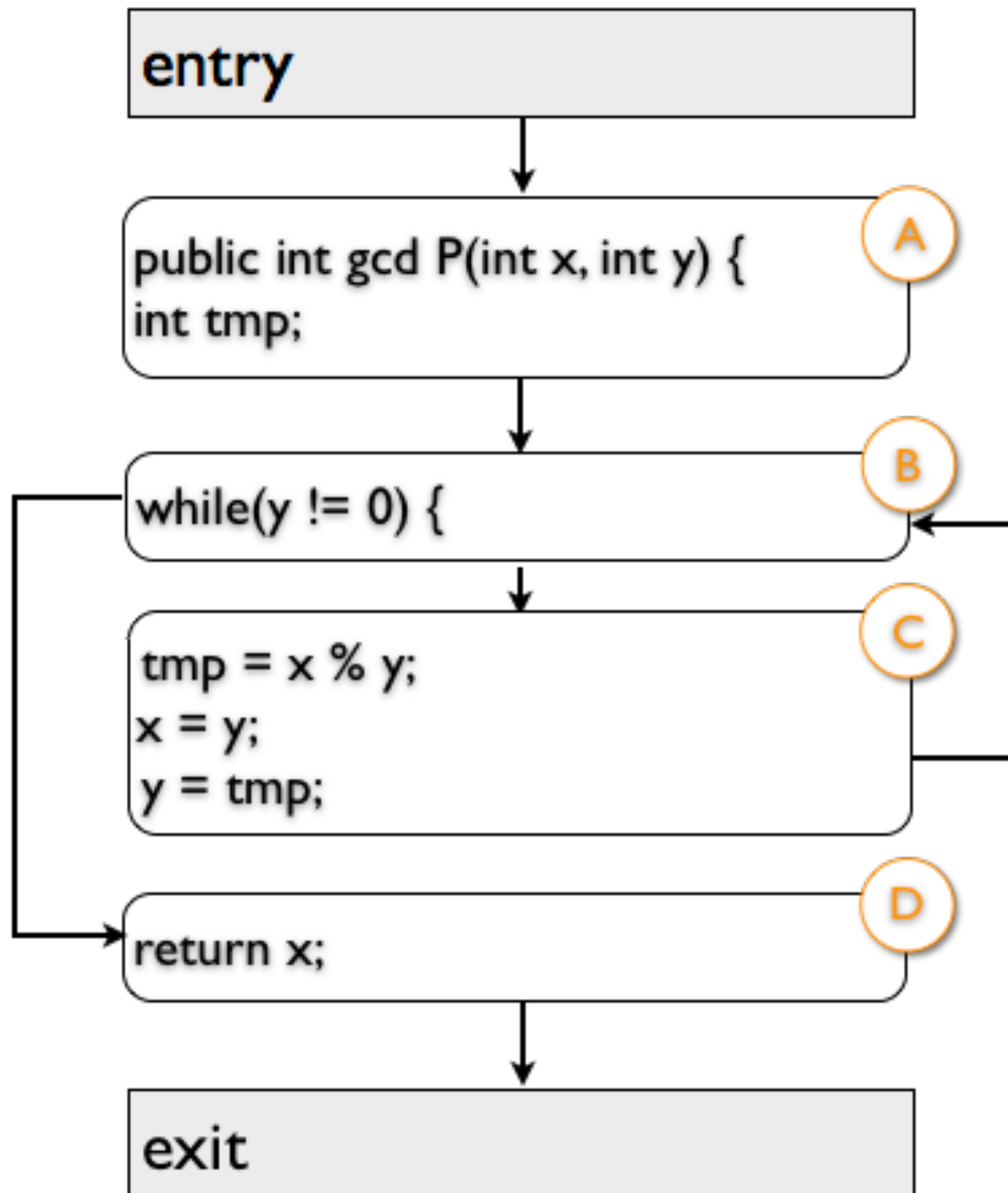
$\text{ReachOut}(C) = \{x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(D) = \{x_A, y_A, \text{tmp}_A\}$

$\text{ReachOut}(D) = \{x_A, y_A, \text{tmp}_A\}$

$\text{worklist} = \{B, C, D, B\}$

$n = D$



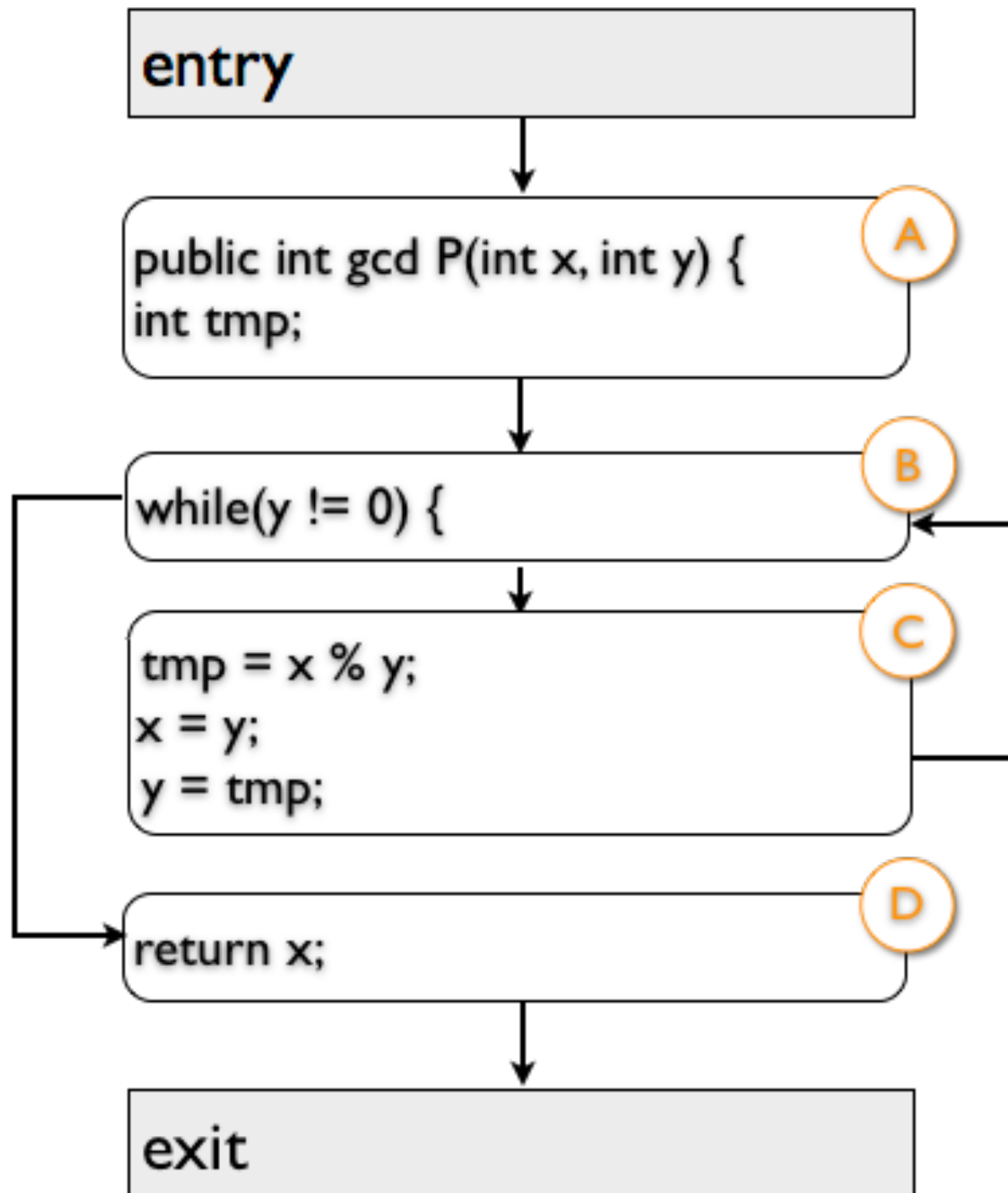
$\text{Reach}(A) = \{\}$
 $\text{ReachOut}(A) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(B) = \{x_A, y_A, \text{tmp}_A\}$
 $\text{ReachOut}(B) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(C) = \{x_A, y_A, \text{tmp}_A\}$
 $\text{ReachOut}(C) = \{x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(D) = \{x_A, y_A, \text{tmp}_A\}$
 $\text{ReachOut}(D) = \{x_A, y_A, \text{tmp}_A\}$

worklist = {B, C, D, B}



$\text{Reach}(A) = \{\}$

$\text{ReachOut}(A) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(B) = \{x_A, y_A, \text{tmp}_A\}$

$\text{ReachOut}(B) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(C) = \{x_A, y_A, \text{tmp}_A\}$

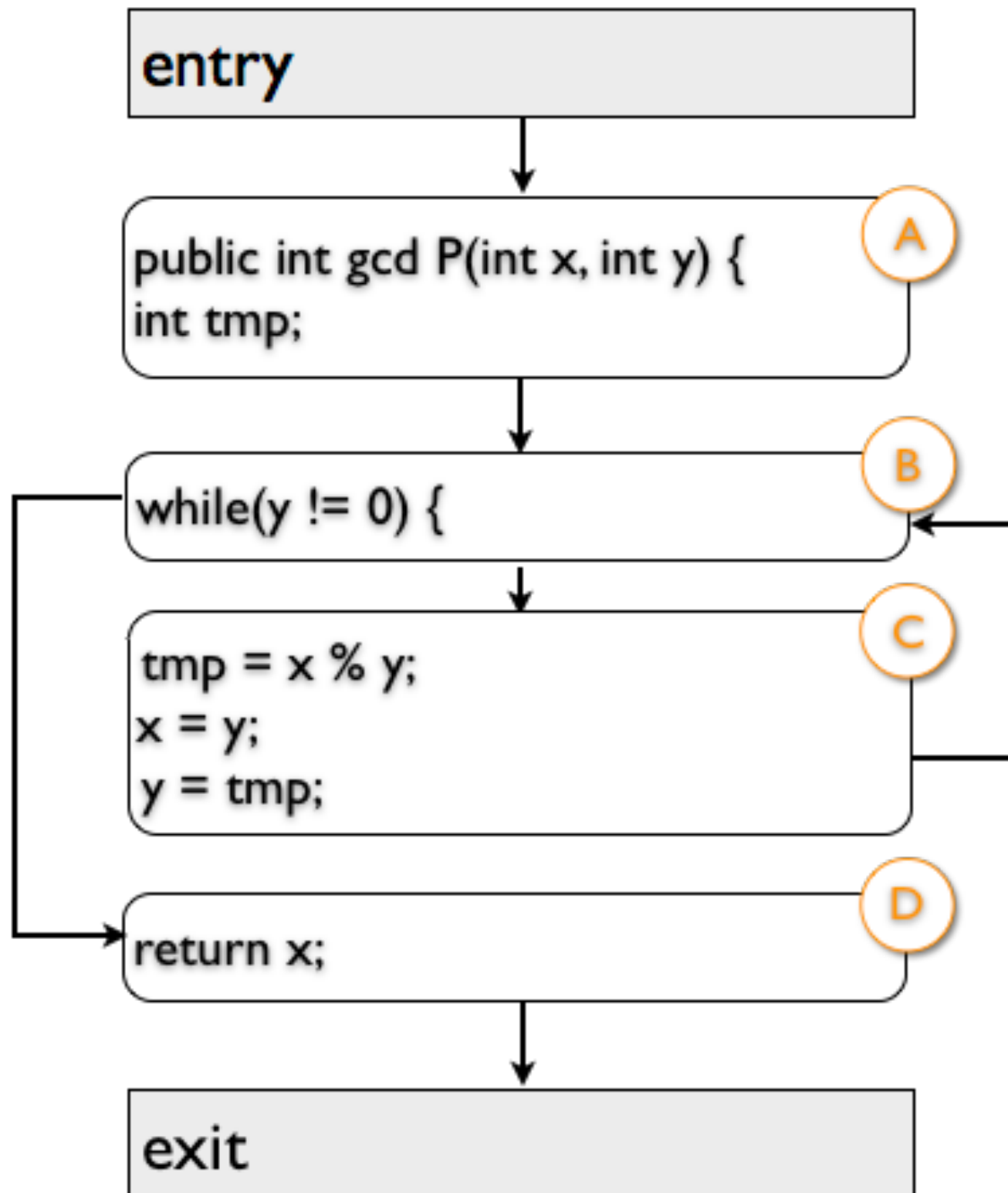
$\text{ReachOut}(C) = \{x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(D) = \{x_A, y_A, \text{tmp}_A\}$

$\text{ReachOut}(D) = \{x_A, y_A, \text{tmp}_A\}$

worklist = {C, D, B}

$n = B$



Reach(A) = {}

ReachOut(A) = { x_A, y_A, tmp_A }

Reach(B) = { $x_A, y_A, tmp_A,$
 x_C, y_C, tmp_C }

ReachOut(B) = { $x_A, y_A, tmp_A,$
 x_C, y_C, tmp_C }

Reach(C) = { x_A, y_A, tmp_A }

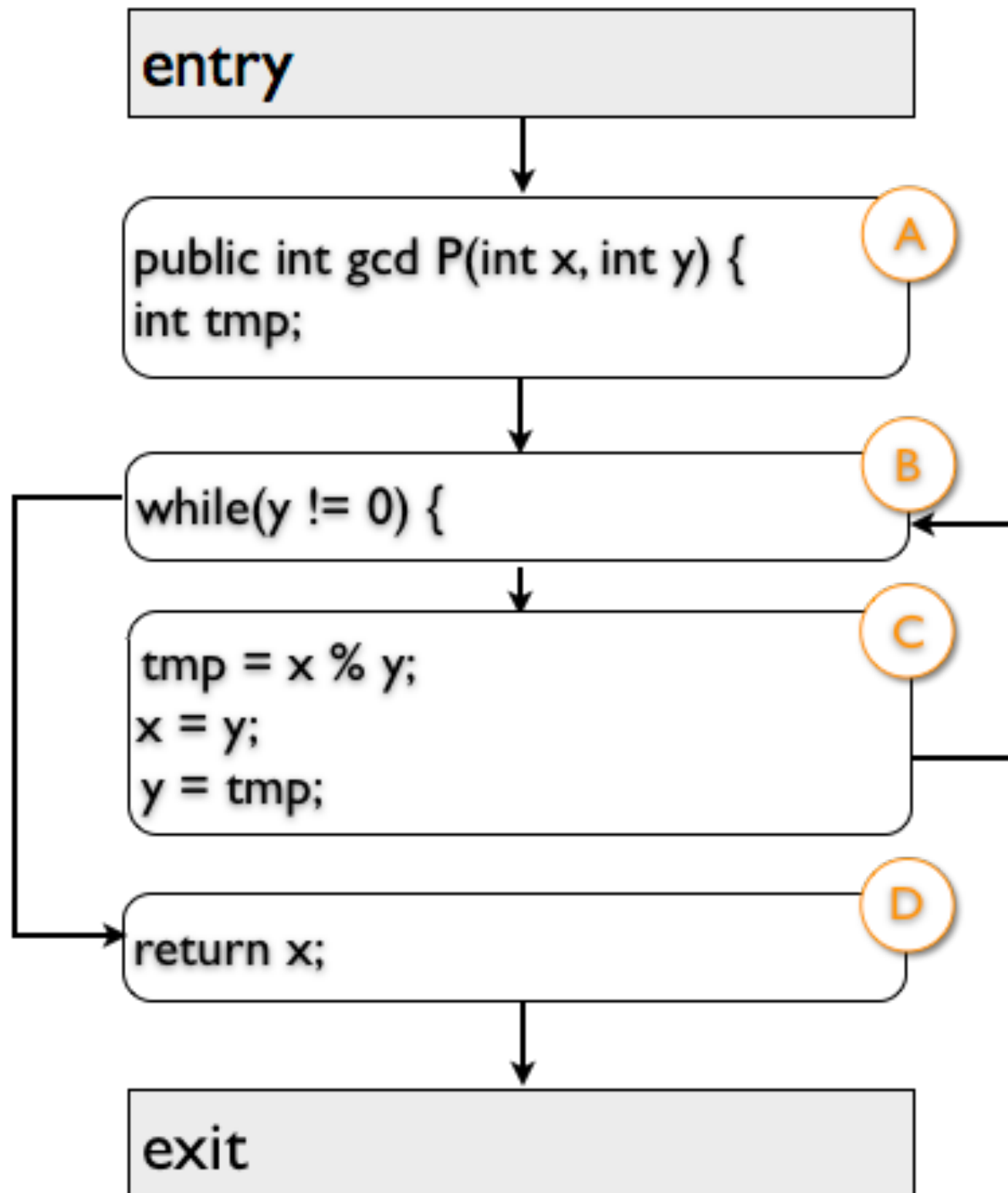
ReachOut(C) = { x_C, y_C, tmp_C }

Reach(D) = { x_A, y_A, tmp_A }

ReachOut(D) = { x_A, y_A, tmp_A }

worklist = {C, D, B}

n = B



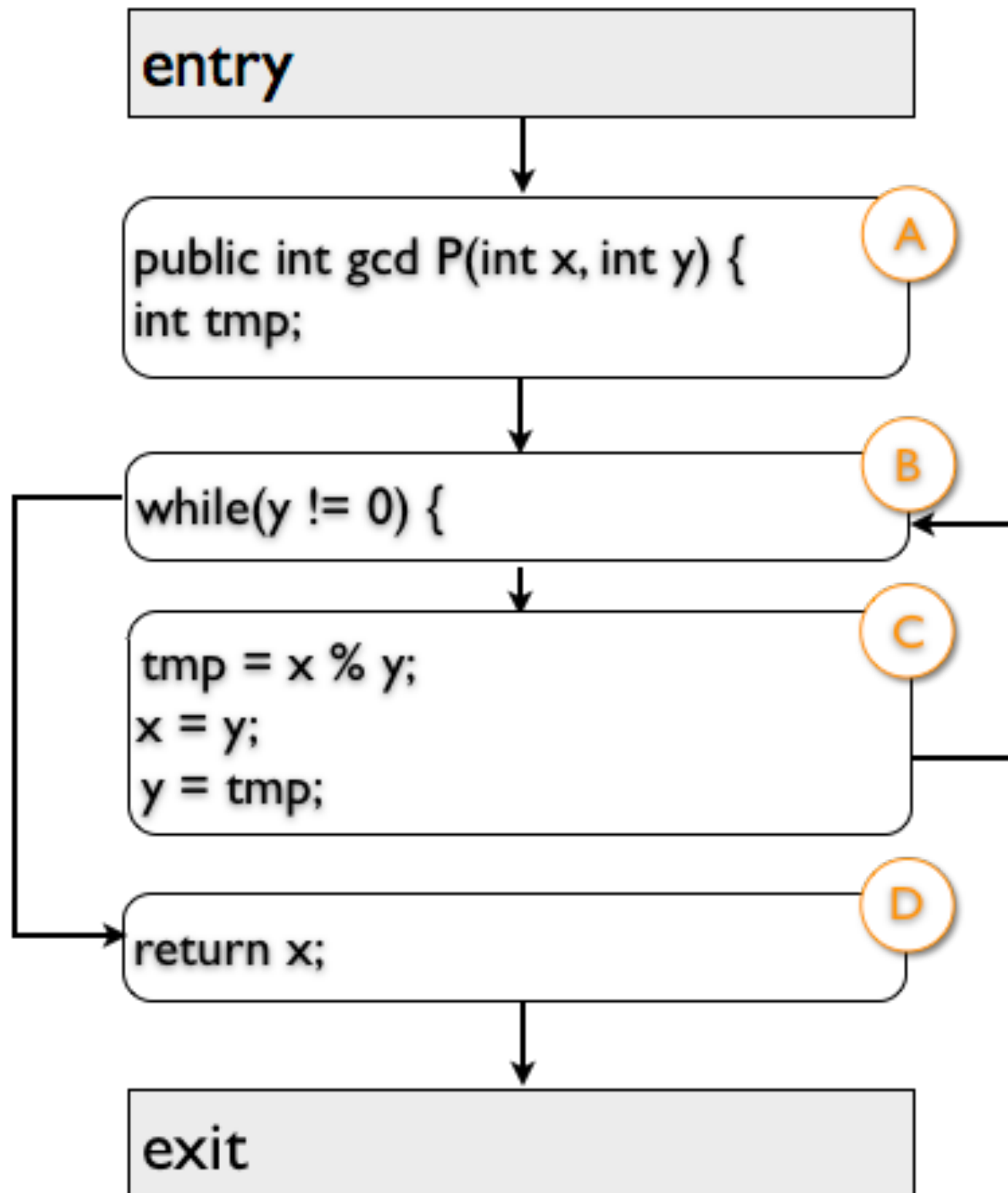
Reach(A) = {}
ReachOut(A) = {x_A, y_A, tmp_A}

Reach(B) = {x_A, y_A, tmp_A,
x_C, y_C, tmp_C}
ReachOut(B) = {x_A, y_A, tmp_A,
x_C, y_C, tmp_C}

Reach(C) = {x_A, y_A, tmp_A}
ReachOut(C) = {x_C, y_C, tmp_C}

Reach(D) = {x_A, y_A, tmp_A}
ReachOut(D) = {x_A, y_A, tmp_A}

worklist = {C, D, B, C, D}



$\text{Reach}(A) = \{\}$

$\text{ReachOut}(A) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(B) = \{x_A, y_A, \text{tmp}_A, \\ x_C, y_C, \text{tmp}_C\}$

$\text{ReachOut}(B) = \{x_A, y_A, \text{tmp}_A, \\ x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(C) = \{x_A, y_A, \text{tmp}_A\}$

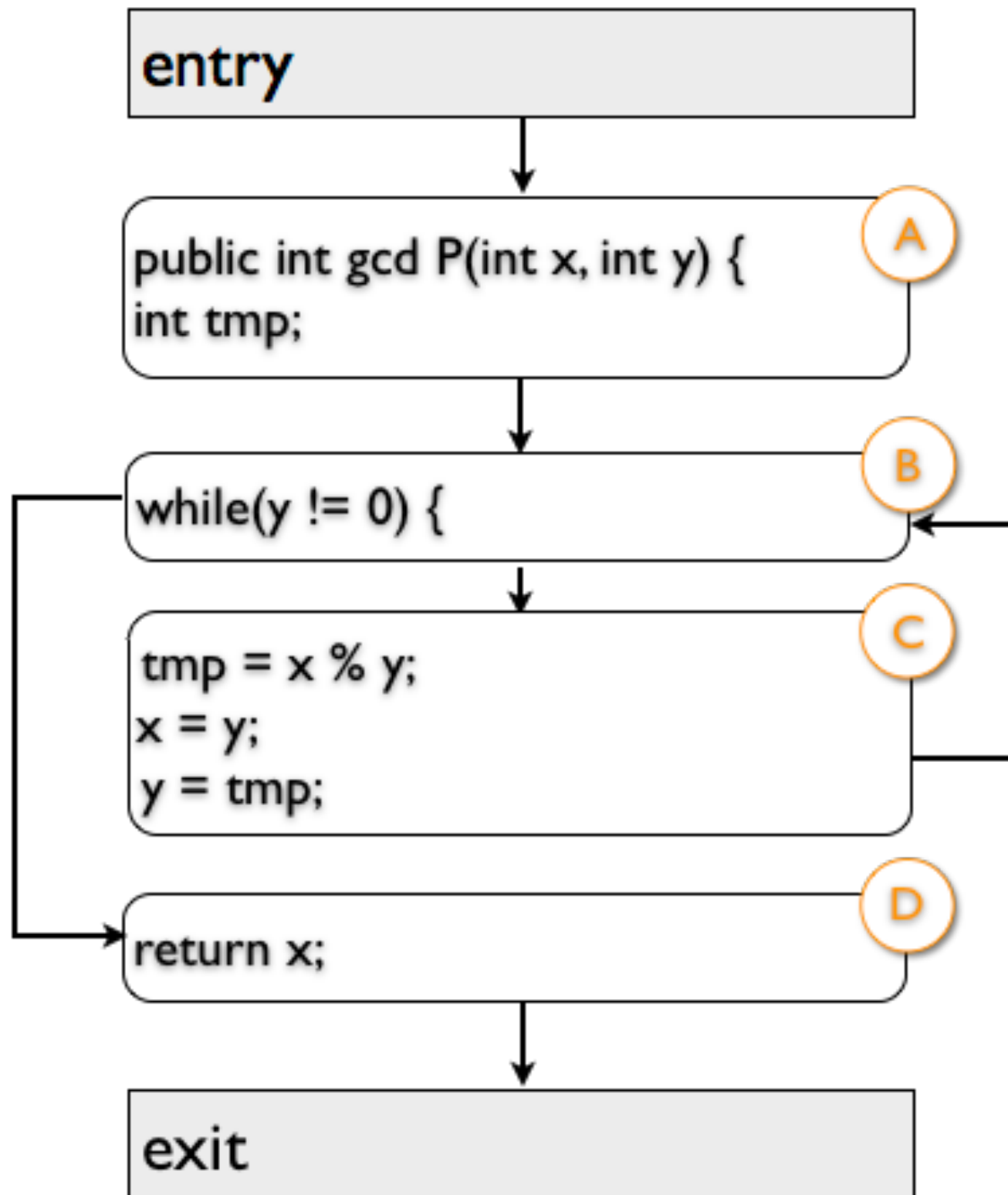
$\text{ReachOut}(C) = \{x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(D) = \{x_A, y_A, \text{tmp}_A\}$

$\text{ReachOut}(D) = \{x_A, y_A, \text{tmp}_A\}$

$\text{worklist} = \{D, B, C, D\}$

$n = C$



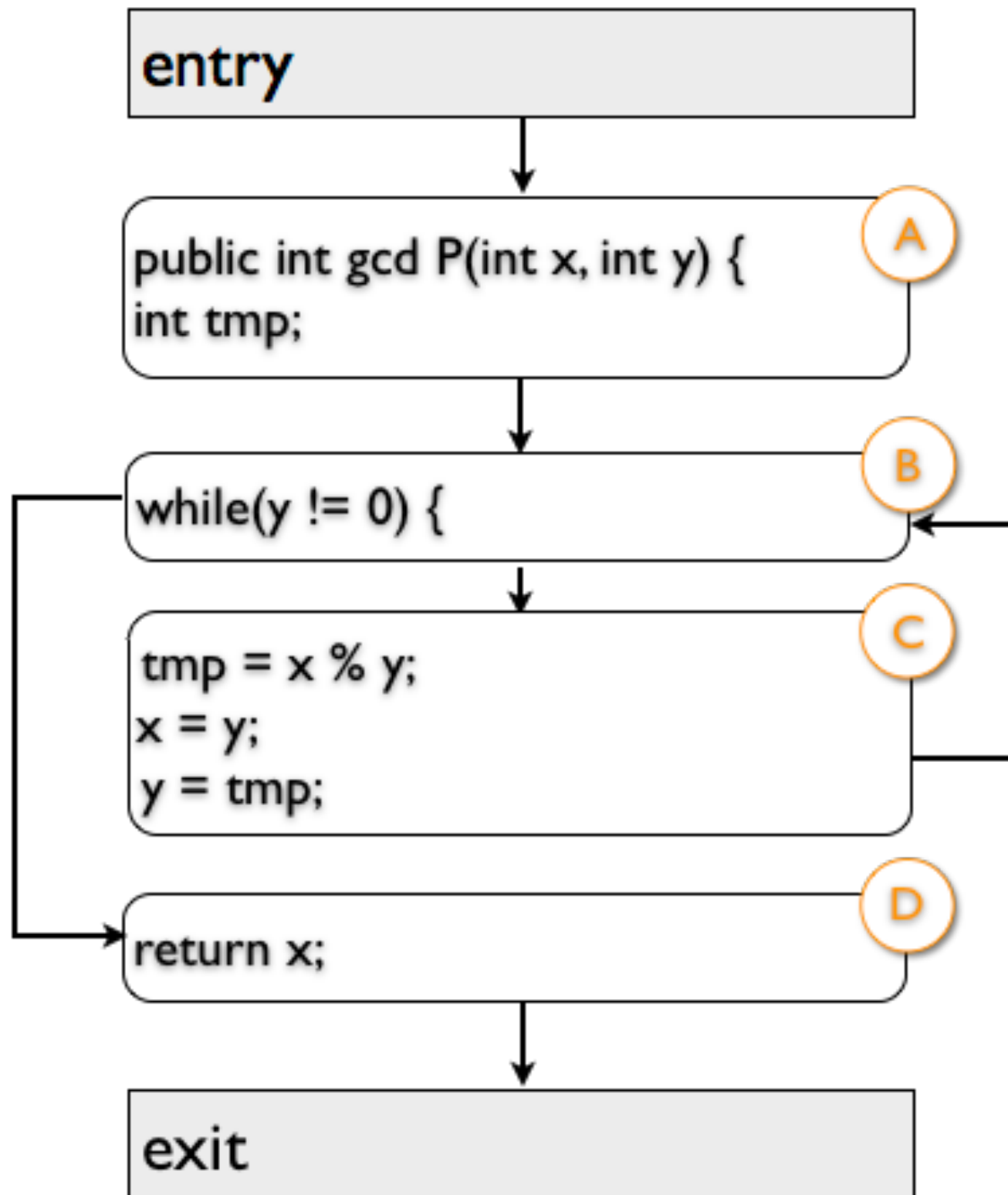
$\text{Reach}(A) = \{\}$
 $\text{ReachOut}(A) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(B) = \{x_A, y_A, \text{tmp}_A, x_C, y_C, \text{tmp}_C\}$
 $\text{ReachOut}(B) = \{x_A, y_A, \text{tmp}_A, x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(C) = \{x_A, y_A, \text{tmp}_A, x_C, y_C, \text{tmp}_C\}$
 $\text{ReachOut}(C) = \{x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(D) = \{x_A, y_A, \text{tmp}_A\}$
 $\text{ReachOut}(D) = \{x_A, y_A, \text{tmp}_A\}$

$\text{worklist} = \{D, B, C, D\}$



$\text{Reach}(A) = \{\}$
 $\text{ReachOut}(A) = \{x_A, y_A, \text{tmp}_A\}$

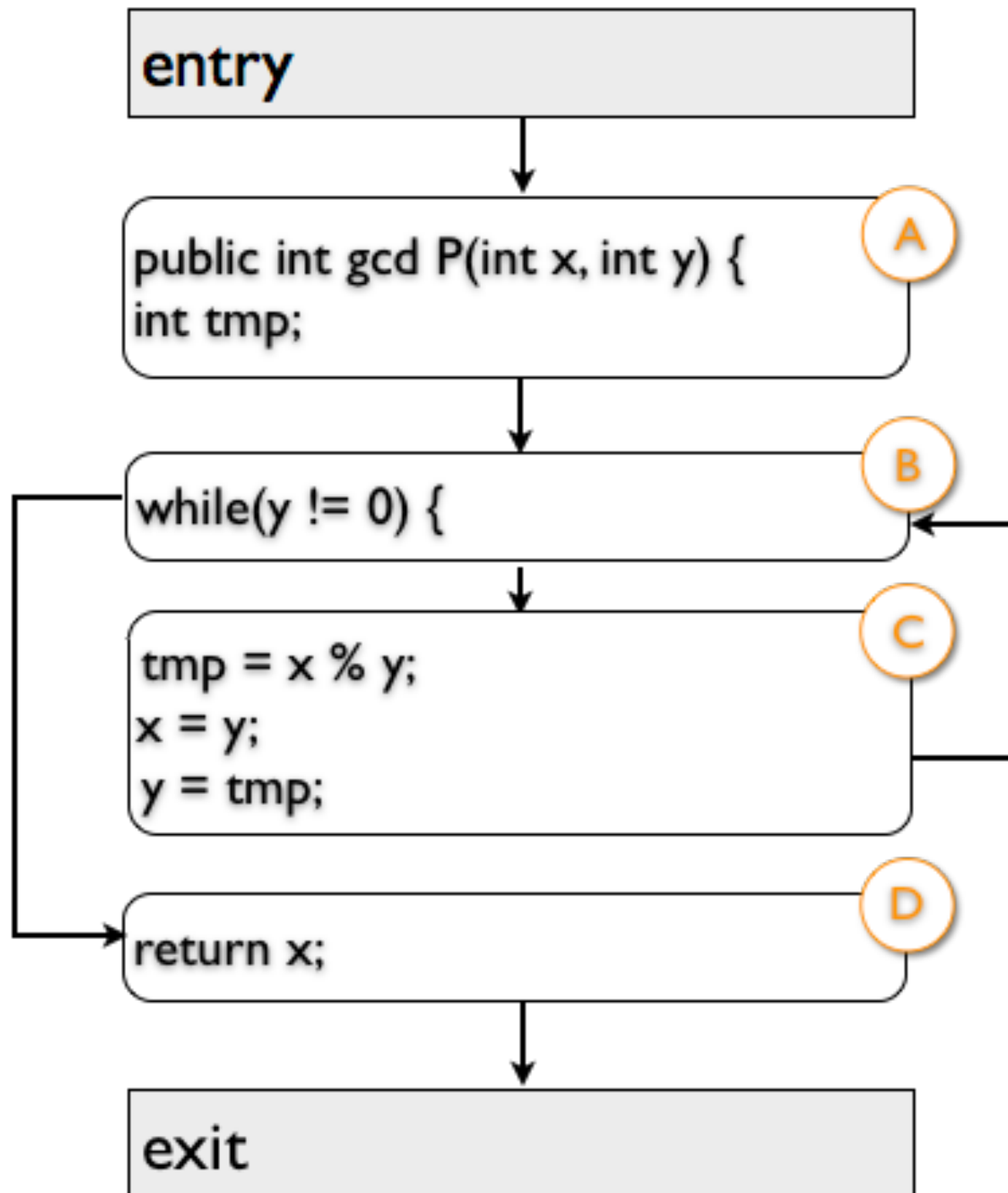
$\text{Reach}(B) = \{x_A, y_A, \text{tmp}_A, x_C, y_C, \text{tmp}_C\}$
 $\text{ReachOut}(B) = \{x_A, y_A, \text{tmp}_A, x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(C) = \{x_A, y_A, \text{tmp}_A, x_C, y_C, \text{tmp}_C\}$
 $\text{ReachOut}(C) = \{x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(D) = \{x_A, y_A, \text{tmp}_A\}$
 $\text{ReachOut}(D) = \{x_A, y_A, \text{tmp}_A\}$

worklist = {B, C, D}

$n = D$



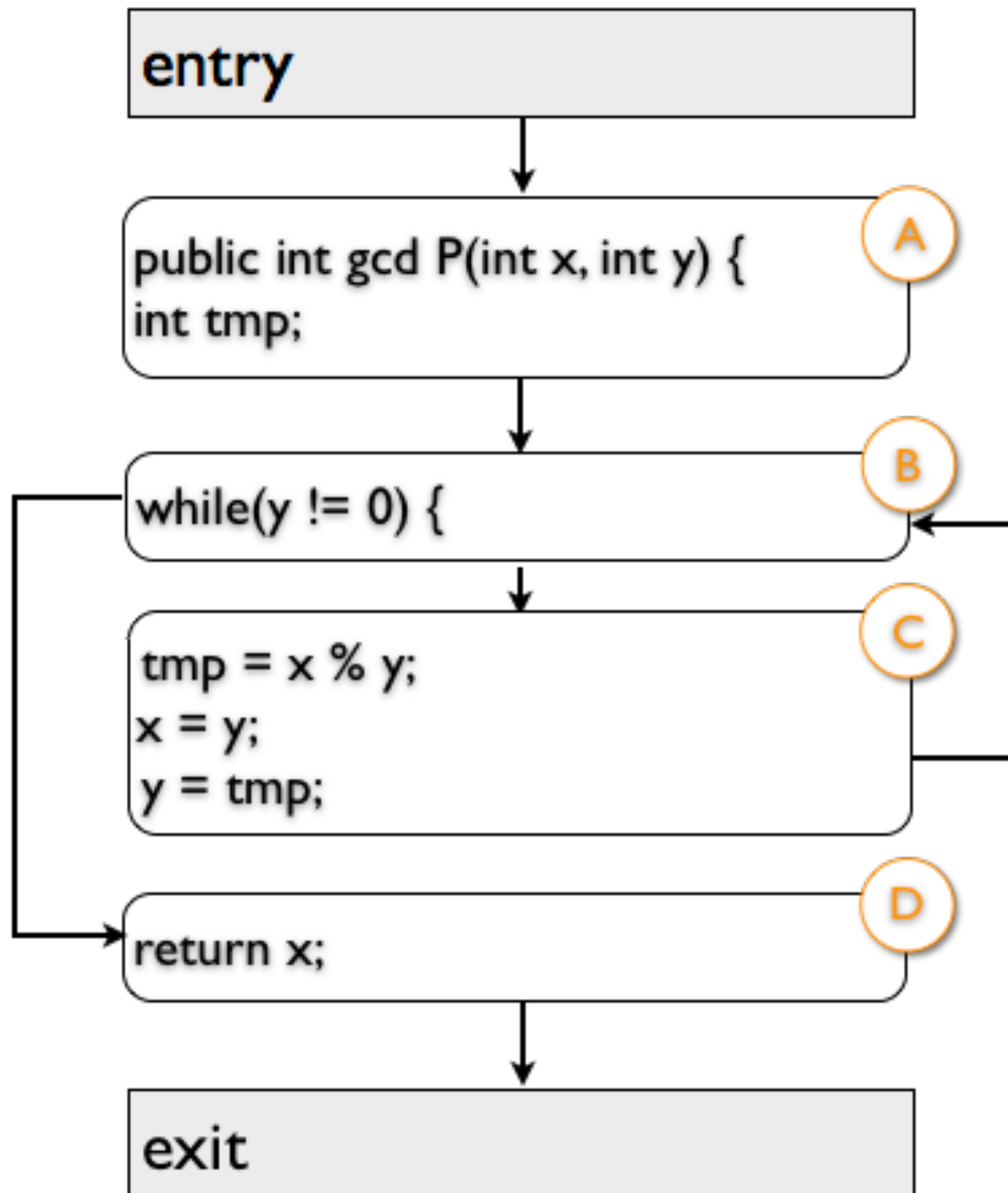
$\text{Reach}(A) = \{\}$
 $\text{ReachOut}(A) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(B) = \{x_A, y_A, \text{tmp}_A, x_C, y_C, \text{tmp}_C\}$
 $\text{ReachOut}(B) = \{x_A, y_A, \text{tmp}_A, x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(C) = \{x_A, y_A, \text{tmp}_A, x_C, y_C, \text{tmp}_C\}$
 $\text{ReachOut}(C) = \{x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(D) = \{x_C, y_C, \text{tmp}_C, x_A, y_A, \text{tmp}_A\}$
 $\text{ReachOut}(D) = \{x_C, y_C, \text{tmp}_C, x_A, y_A, \text{tmp}_A\}$

$\text{worklist} = \{B, C, D\}$
 $n = D$



$\text{Reach}(A) = \{\}$

$\text{ReachOut}(A) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(B) = \{x_A, y_A, \text{tmp}_A, \\ x_C, y_C, \text{tmp}_C\}$

$\text{ReachOut}(B) = \{x_A, y_A, \text{tmp}_A, \\ x_C, y_C, \text{tmp}_C\}$

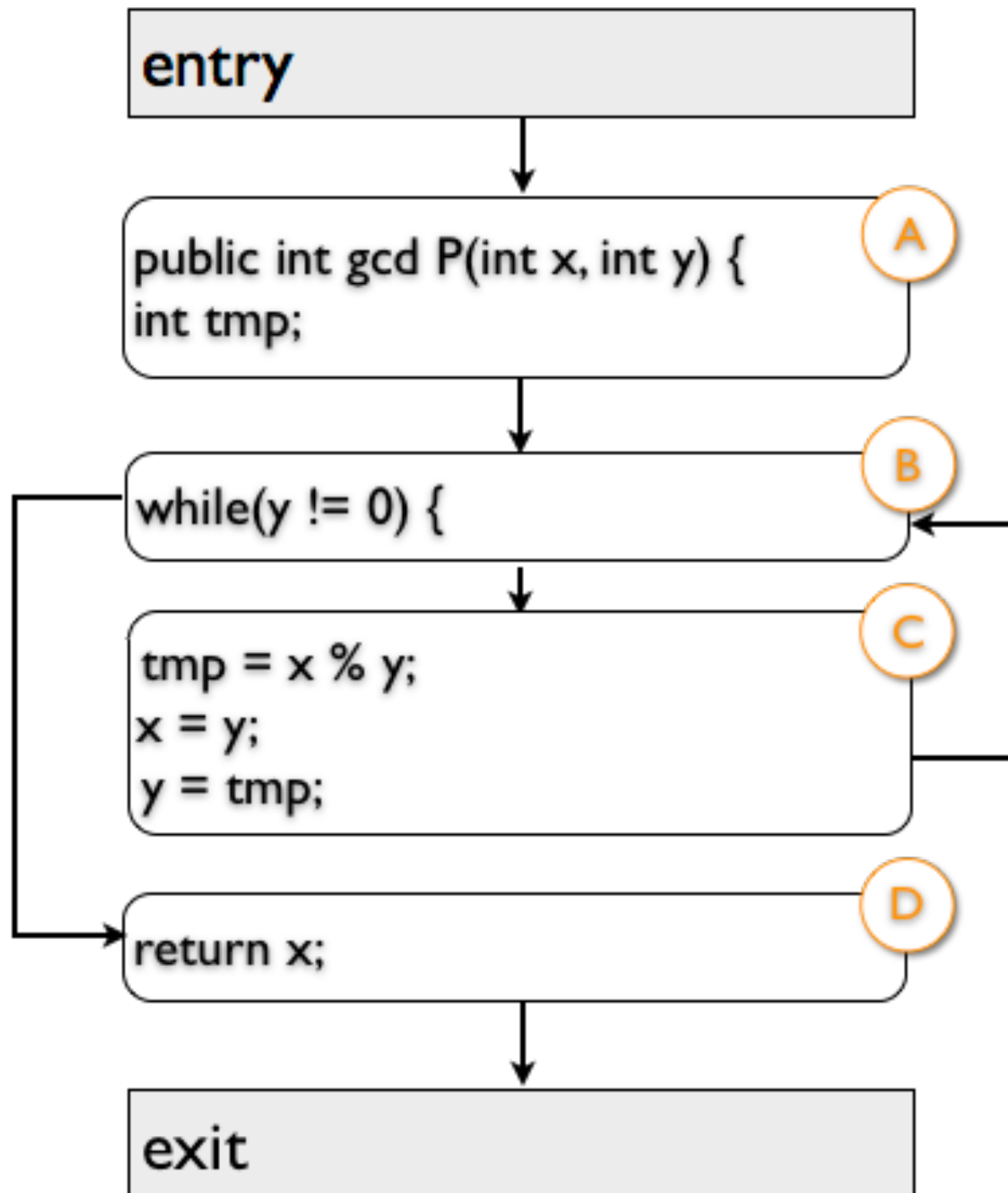
$\text{Reach}(C) = \{x_A, y_A, \text{tmp}_A, \\ x_C, y_C, \text{tmp}_C\}$

$\text{ReachOut}(C) = \{x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(D) = \{x_C, y_C, \text{tmp}_C, \\ x_A, y_A, \text{tmp}_A\}$

$\text{ReachOut}(D) = \{x_C, y_C, \text{tmp}_C, \\ x_A, y_A, \text{tmp}_A\}$

$\text{worklist} = \{B, C, D\}$



Reach(A) = {}

ReachOut(A) = {x_A, y_A, tmp_A}

Reach(B) = {x_A, y_A, tmp_A,
x_C, y_C, tmp_C}

ReachOut(B) = {x_A, y_A, tmp_A,
x_C, y_C, tmp_C}

Reach(C) = {x_A, y_A, tmp_A,
x_C, y_C, tmp_C}

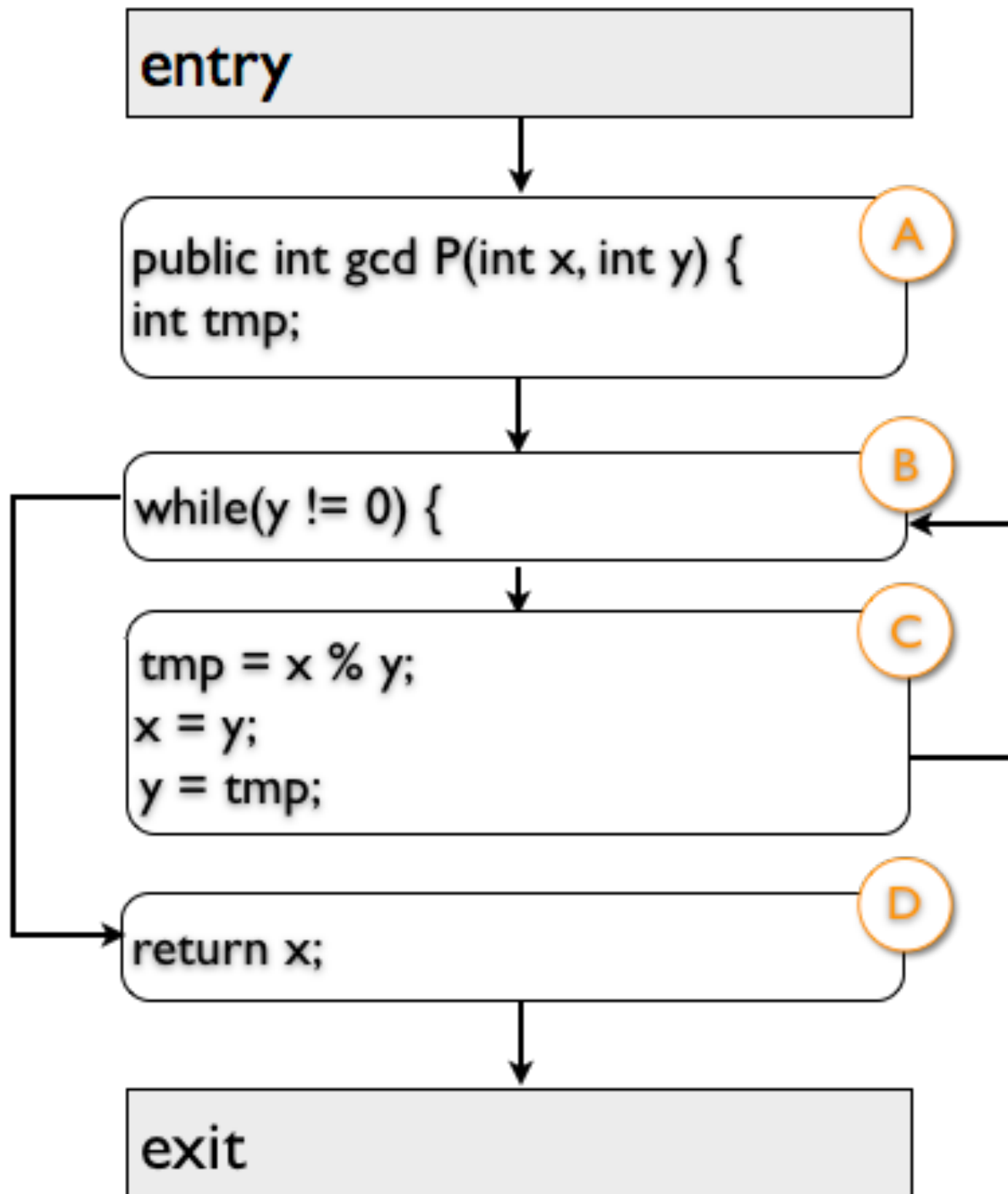
ReachOut(C) = {x_C, y_C, tmp_C}

Reach(D) = {x_C, y_C, tmp_C,
x_A, y_A, tmp_A}

ReachOut(D) = {x_C, y_C, tmp_C,
x_A, y_A, tmp_A}

worklist = {C, D}

n = B



Reach(A) = {}

ReachOut(A) = {x_A, y_A, tmp_A}

Reach(B) = {x_A, y_A, tmp_A,
x_C, y_C, tmp_C}

ReachOut(B) = {x_A, y_A, tmp_A,
x_C, y_C, tmp_C}

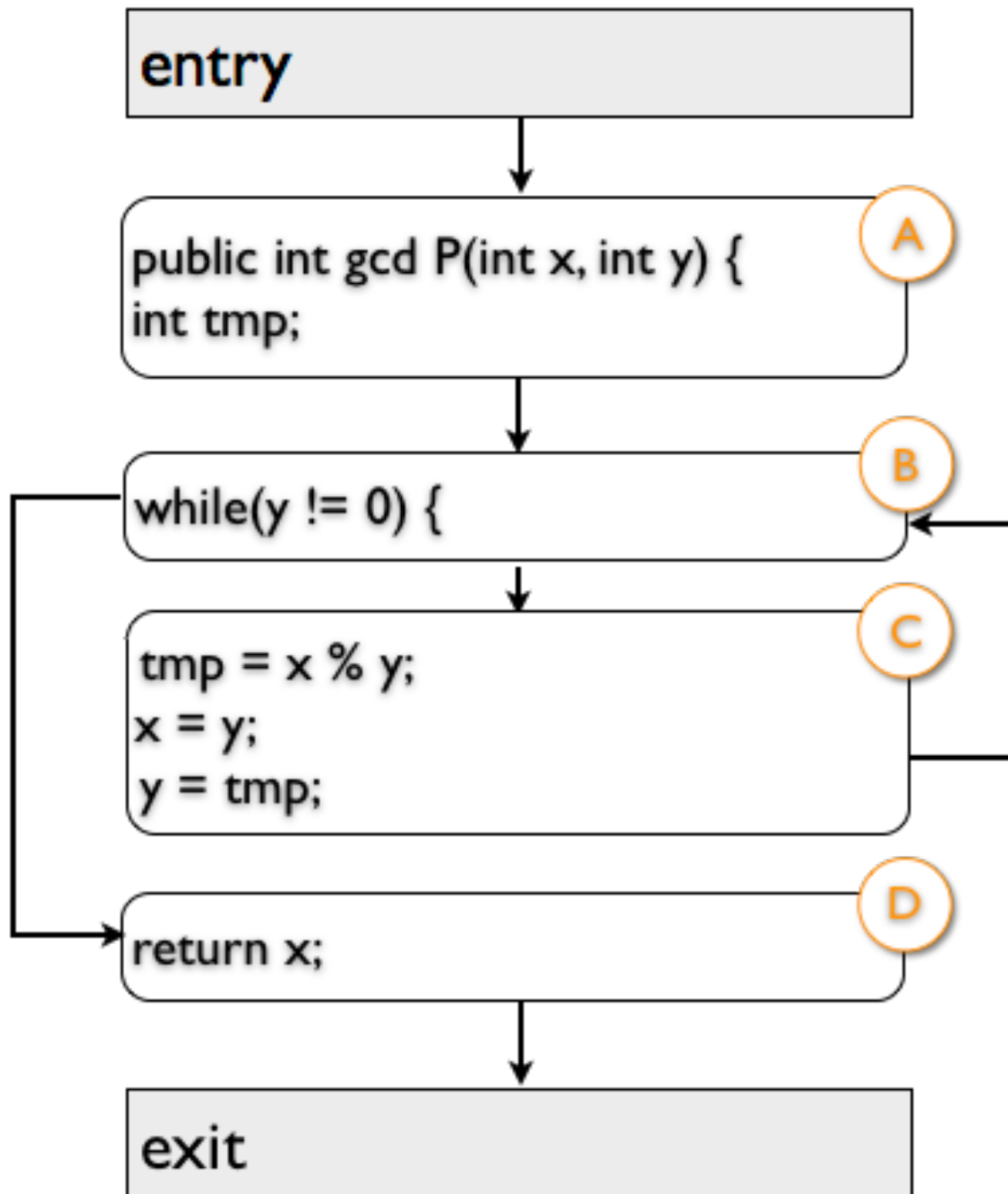
Reach(C) = {x_A, y_A, tmp_A,
x_C, y_C, tmp_C}

ReachOut(C) = {x_C, y_C, tmp_C}

Reach(D) = {x_C, y_C, tmp_C,
x_A, y_A, tmp_A}

ReachOut(D) = {x_C, y_C, tmp_C,
x_A, y_A, tmp_A}

worklist = {C, D}



Reach(A) = {}

ReachOut(A) = {x_A, y_A, tmp_A}

Reach(B) = {x_A, y_A, tmp_A,
x_C, y_C, tmp_C}

ReachOut(B) = {x_A, y_A, tmp_A,
x_C, y_C, tmp_C}

Reach(C) = {x_A, y_A, tmp_A,
x_C, y_C, tmp_C}

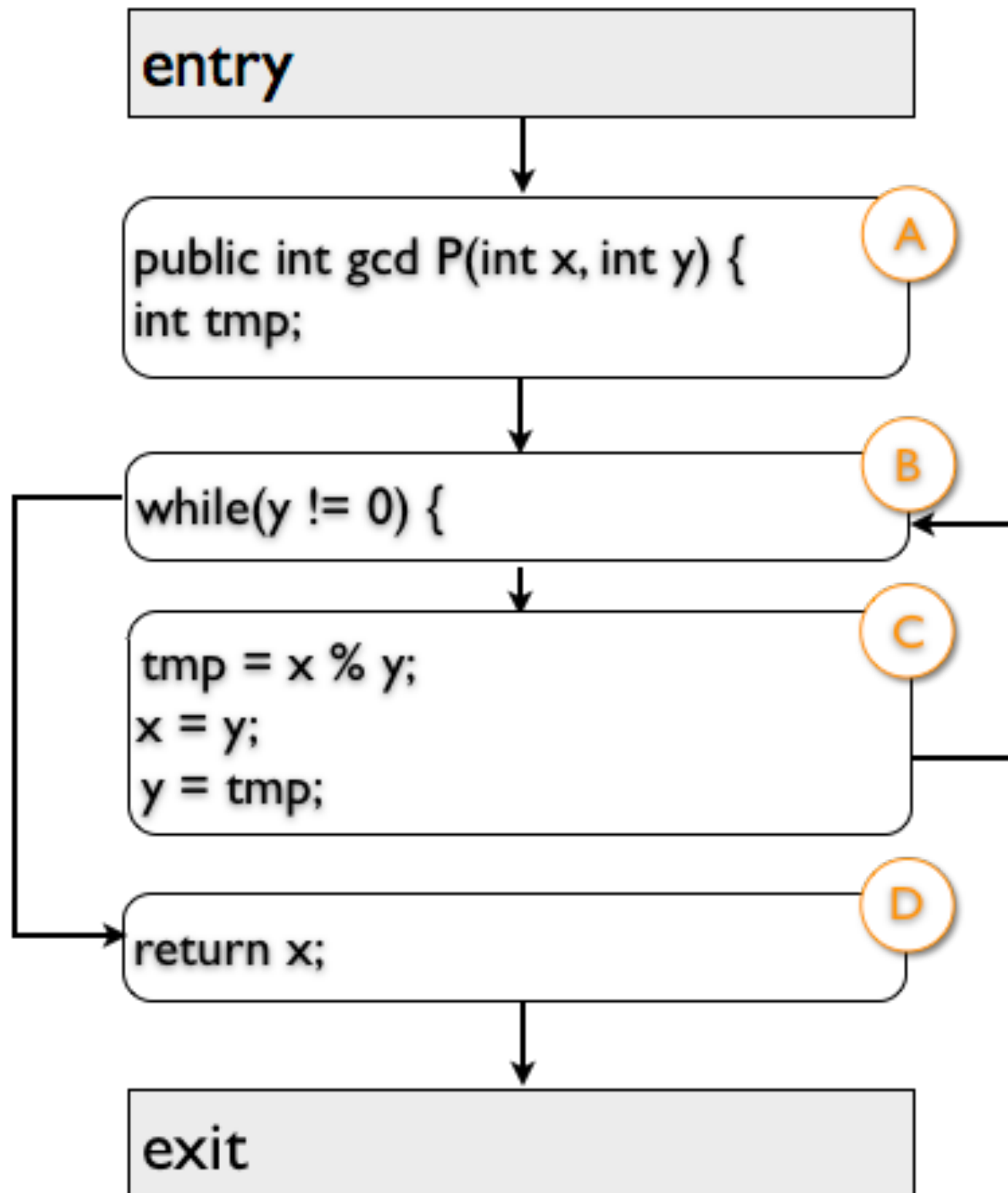
ReachOut(C) = {x_C, y_C, tmp_C}

Reach(D) = {x_C, y_C, tmp_C,
x_A, y_A, tmp_A}

ReachOut(D) = {x_C, y_C, tmp_C,
x_A, y_A, tmp_A}

worklist = {D}

n = C



$\text{Reach}(A) = \{\}$

$\text{ReachOut}(A) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(B) = \{x_A, y_A, \text{tmp}_A, \\ x_C, y_C, \text{tmp}_C\}$

$\text{ReachOut}(B) = \{x_A, y_A, \text{tmp}_A, \\ x_C, y_C, \text{tmp}_C\}$

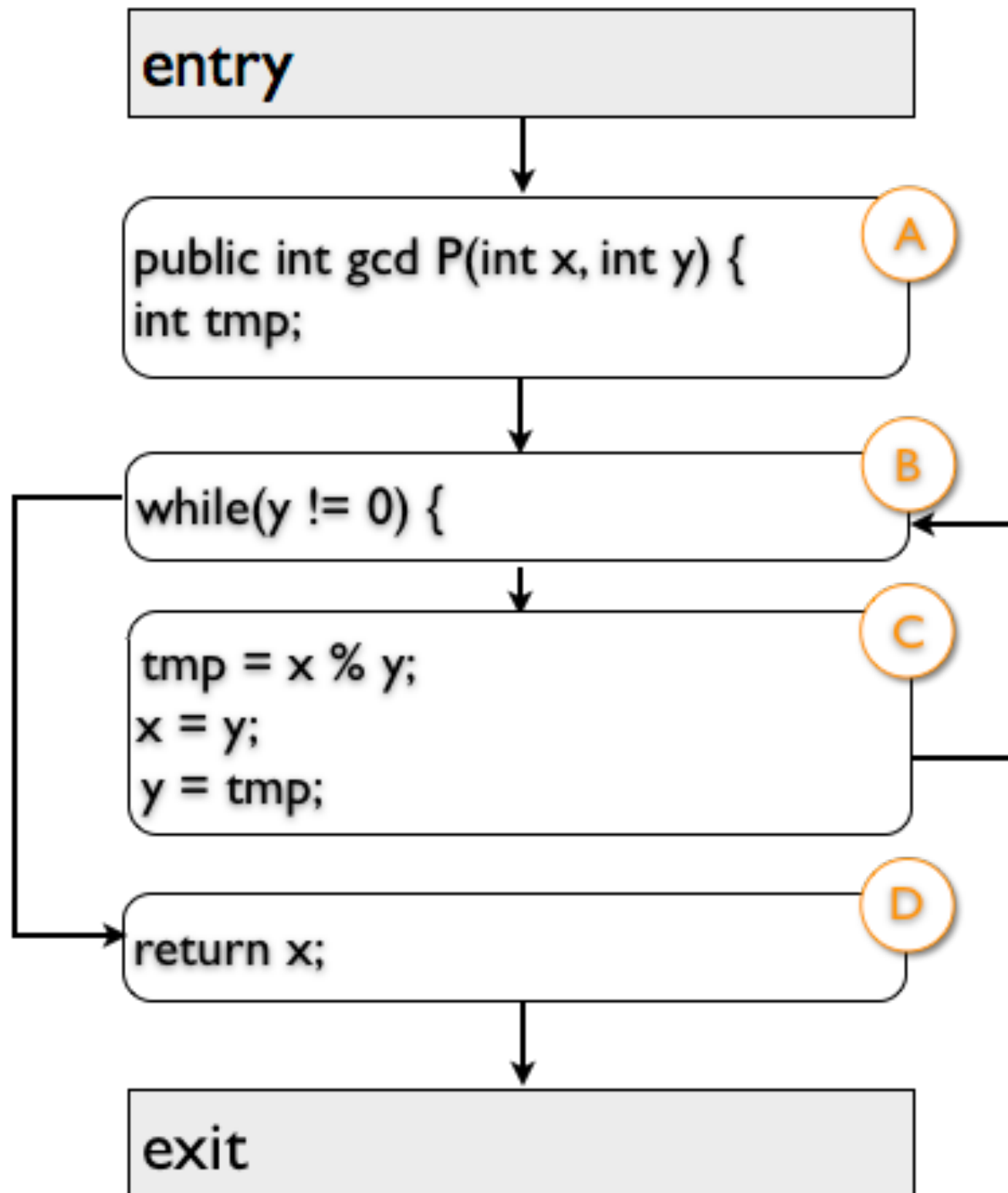
$\text{Reach}(C) = \{x_A, y_A, \text{tmp}_A, \\ x_C, y_C, \text{tmp}_C\}$

$\text{ReachOut}(C) = \{x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(D) = \{x_C, y_C, \text{tmp}_C, \\ x_A, y_A, \text{tmp}_A\}$

$\text{ReachOut}(D) = \{x_C, y_C, \text{tmp}_C, \\ x_A, y_A, \text{tmp}_A\}$

$\text{worklist} = \{D\}$



Reach(A) = {}

ReachOut(A) = {x_A, y_A, tmp_A}

Reach(B) = {x_A, y_A, tmp_A,
x_C, y_C, tmp_C}

ReachOut(B) = {x_A, y_A, tmp_A,
x_C, y_C, tmp_C}

Reach(C) = {x_A, y_A, tmp_A,
x_C, y_C, tmp_C}

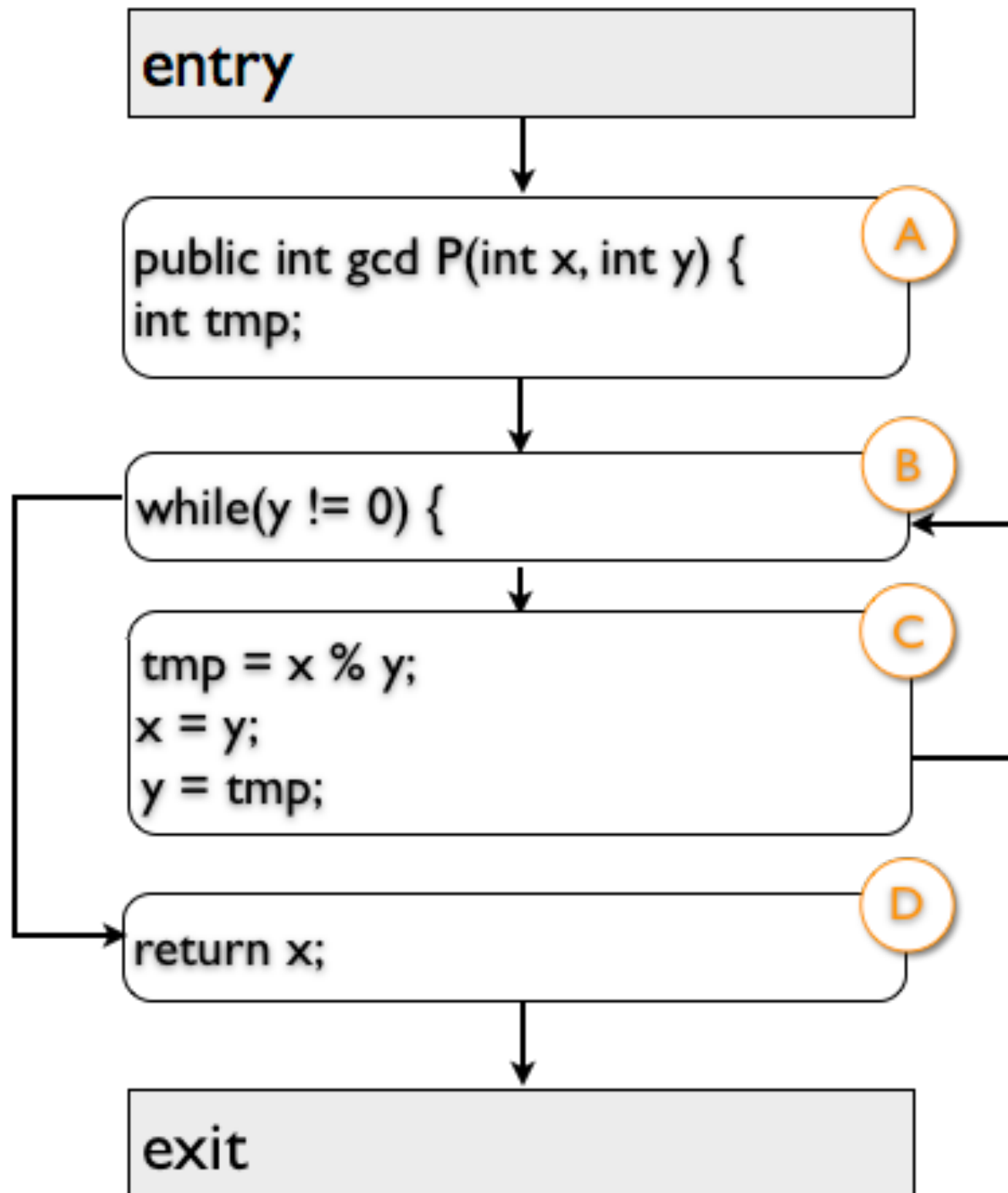
ReachOut(C) = {x_C, y_C, tmp_C}

Reach(D) = {x_C, y_C, tmp_C,
x_A, y_A, tmp_A}

ReachOut(D) = {x_C, y_C, tmp_C,
x_A, y_A, tmp_A}

worklist = {}

n = D



$\text{Reach}(A) = \{\}$

$\text{ReachOut}(A) = \{x_A, y_A, \text{tmp}_A\}$

$\text{Reach}(B) = \{x_A, y_A, \text{tmp}_A, \\ x_C, y_C, \text{tmp}_C\}$

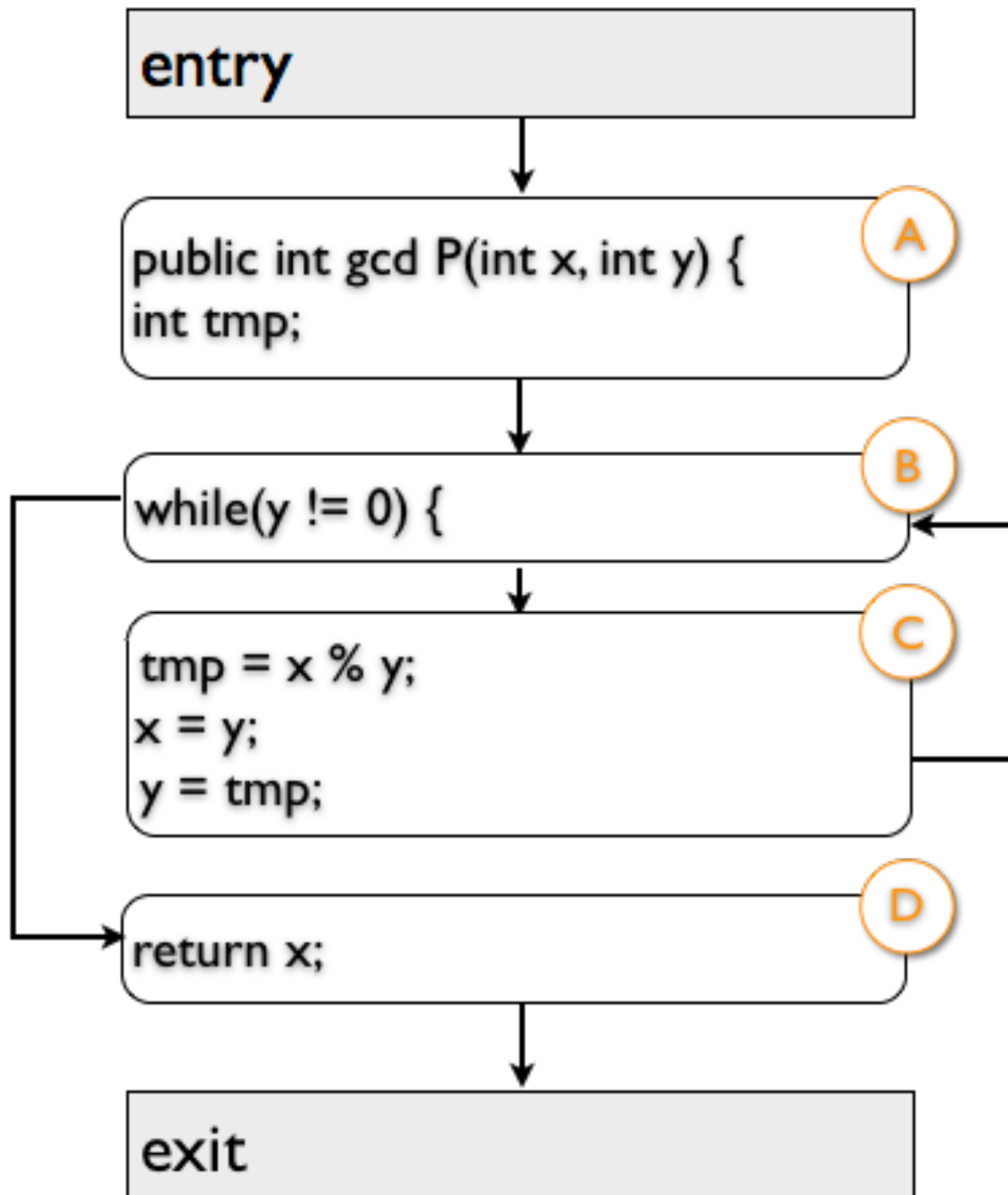
$\text{ReachOut}(B) = \{x_A, y_A, \text{tmp}_A, \\ x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(C) = \{x_A, y_A, \text{tmp}_A, \\ x_C, y_C, \text{tmp}_C\}$

$\text{ReachOut}(C) = \{x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(D) = \{x_C, y_C, \text{tmp}_C, \\ x_A, y_A, \text{tmp}_A\}$

$\text{ReachOut}(D) = \{x_C, y_C, \text{tmp}_C, \\ x_A, y_A, \text{tmp}_A\}$



$\text{Reach}(A) = \{\}$

$\text{Reach}(B) = \{x_A, y_A, \text{tmp}_A, x_C, y_C, \text{tmp}_C\}$

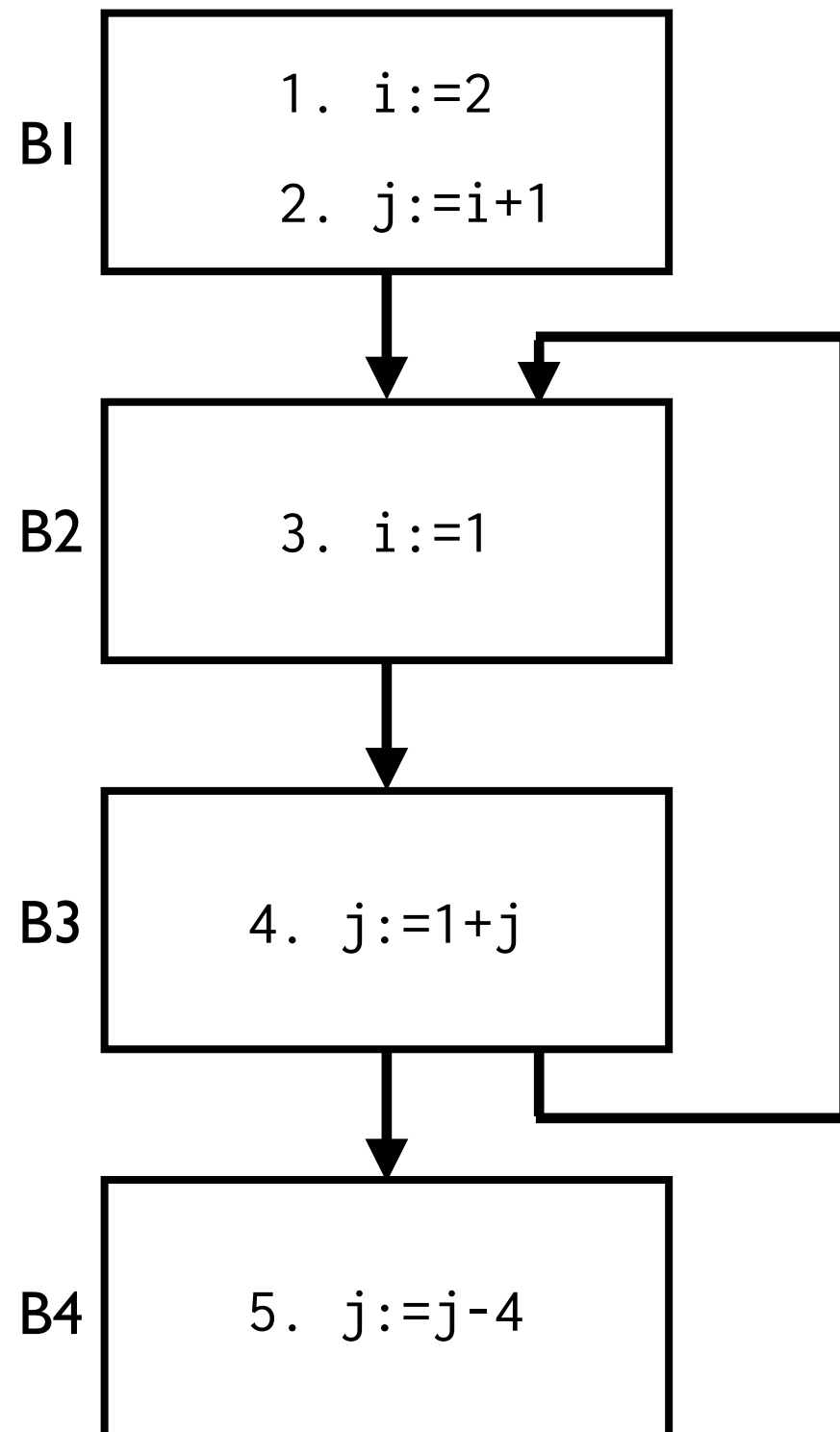
$\text{Reach}(C) = \{x_A, y_A, \text{tmp}_A, x_C, y_C, \text{tmp}_C\}$

$\text{Reach}(D) = \{x_C, y_C, \text{tmp}_C, x_A, y_A, \text{tmp}_A\}$

Outline

1. Reaching Definitions
2. Reachable Uses
3. DU Chains, UD Chains
4. Iterative Dataflow Analysis

Reachable Uses

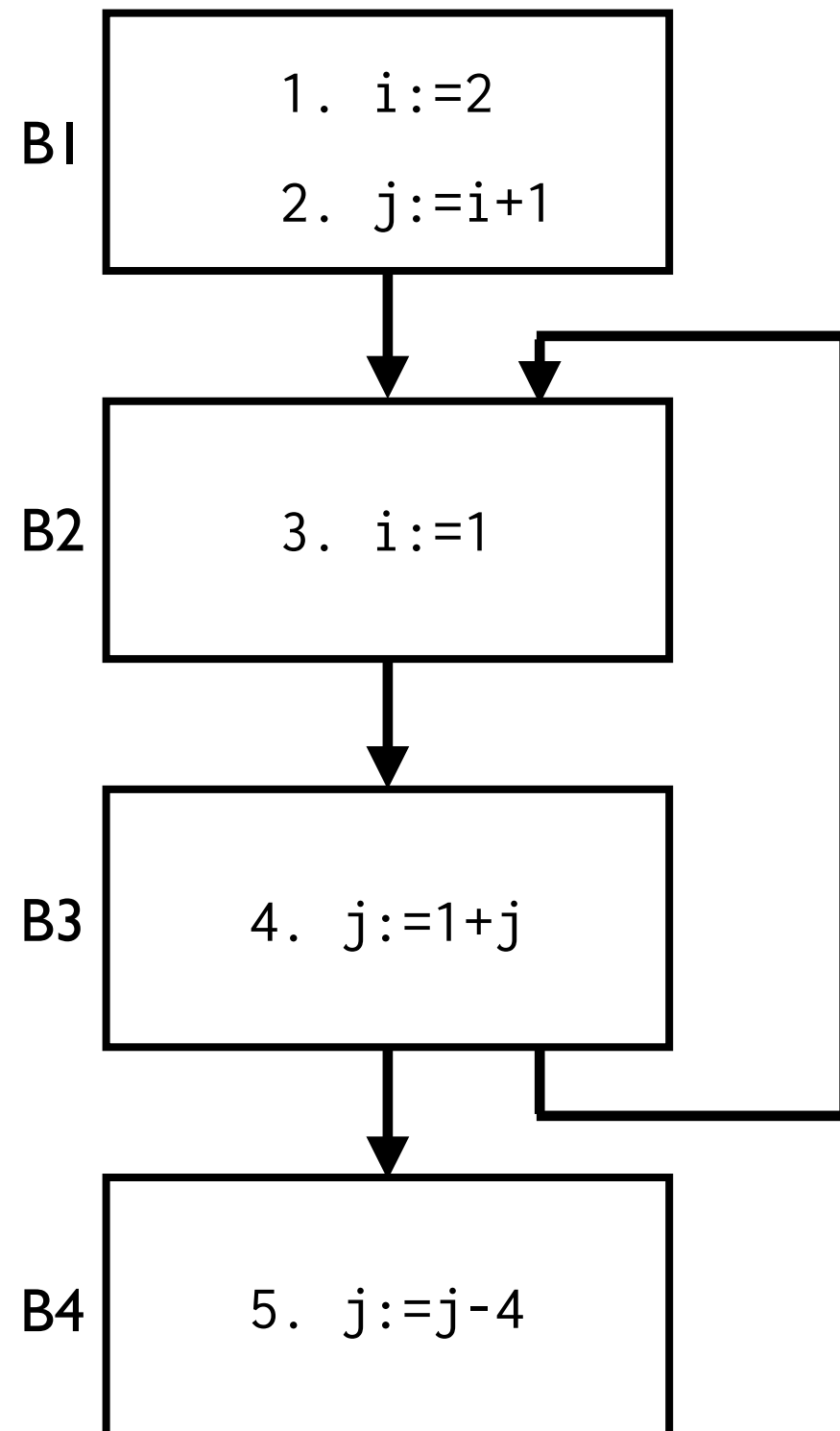


A **use** of a variable or memory location is a point or statement where that variable is referenced but not changed

A **use of A is reachable** from a point p if there exists a control-flow path in the CFG from p to the use with no definitions of A on the path

Reachable uses also called **upwards exposed uses**

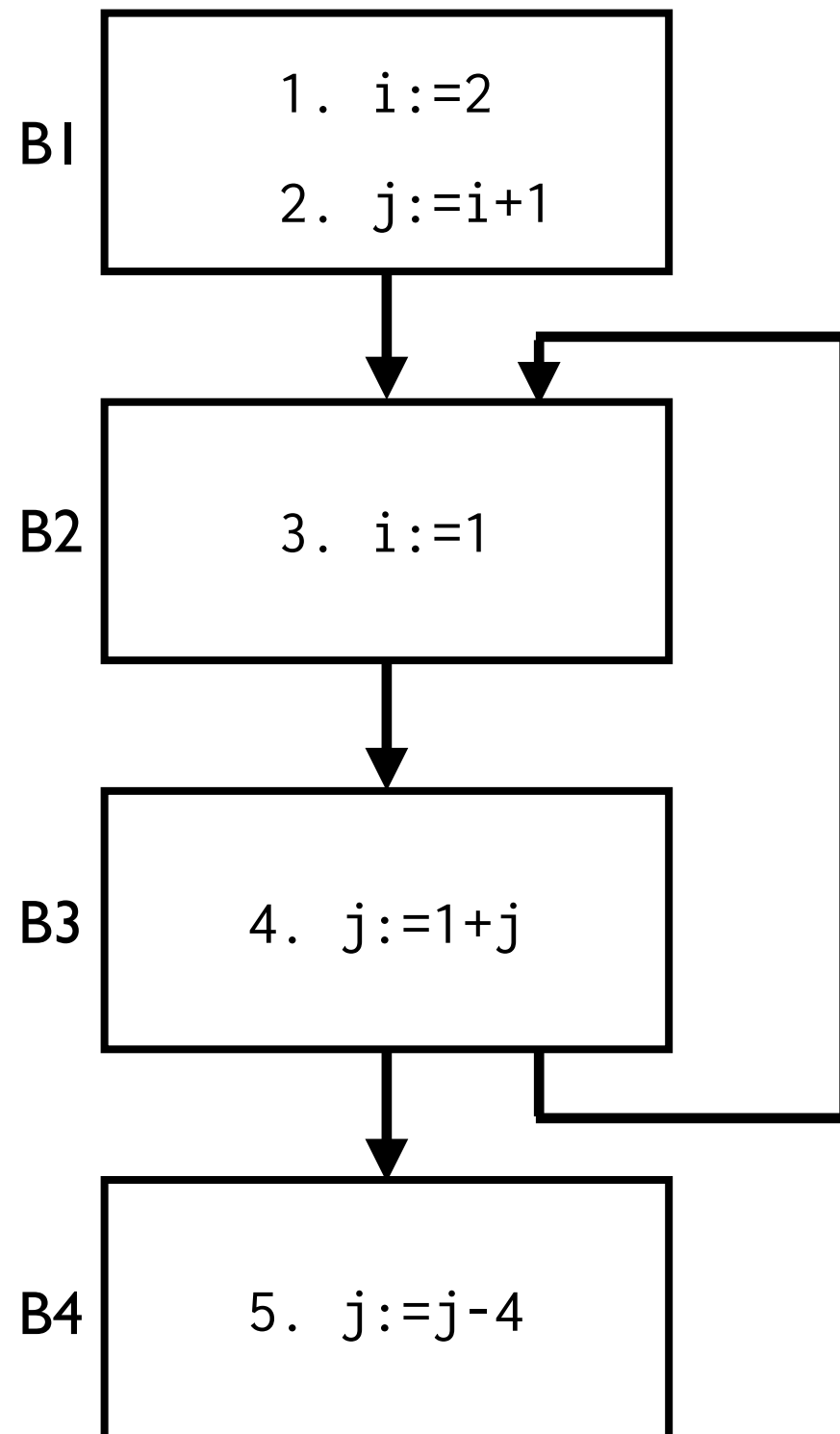
Iterative Dataflow Analysis



Where are the uses in the program?

From which basic blocks (end of) are these uses reachable?

Iterative Dataflow Analysis



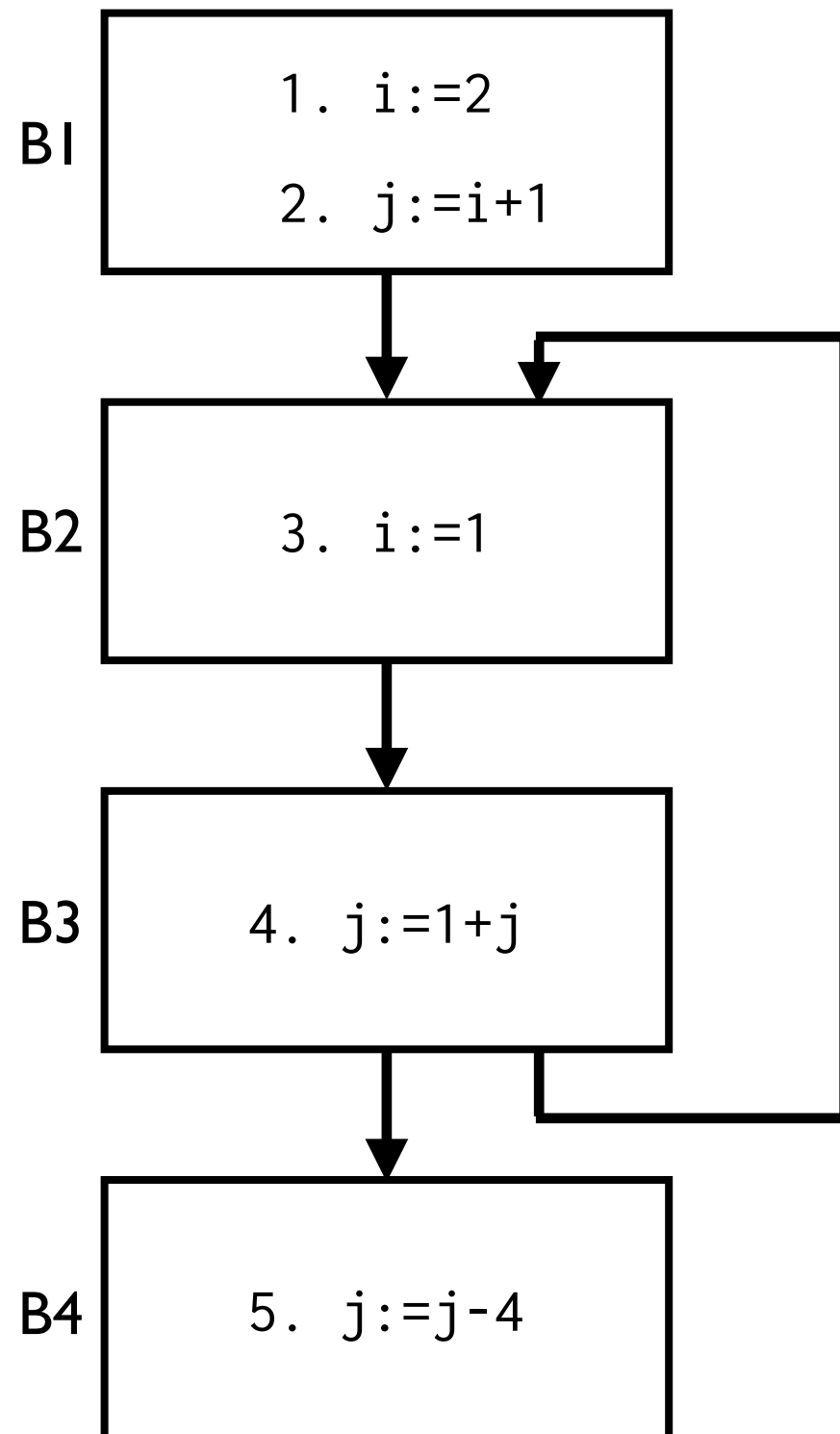
Where are the uses in the program?

- i : 2.1
- j : 4.2, 5.1

From which basic blocks (end of) are these uses reachable?

- Use 4.2 is reachable from B1, B2, B3
- Use 5.1 is reachable from B3

Iterative Dataflow Analysis



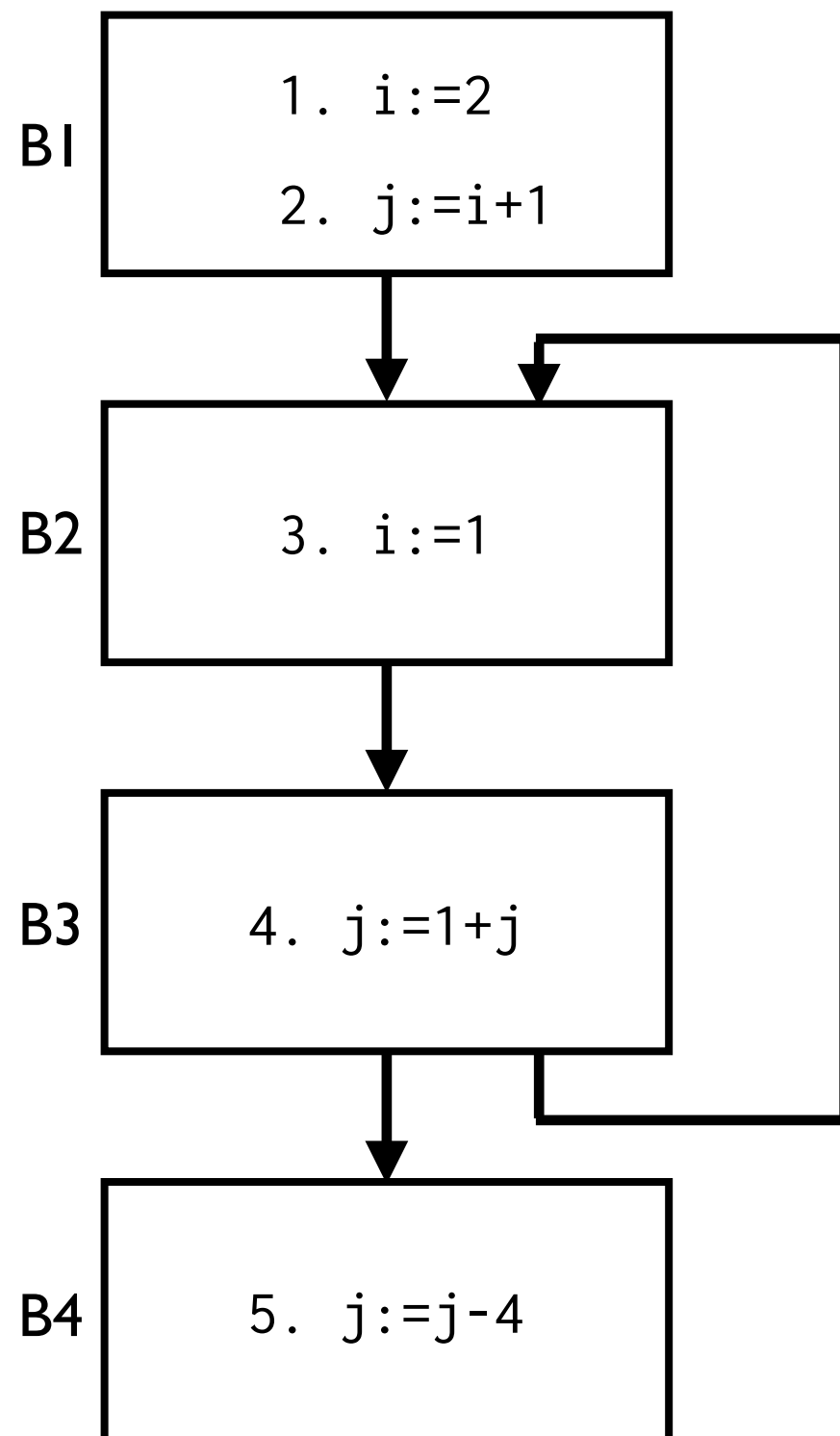
1. Compute two kinds of local information:

- GEN(B): Set of uses created (generated) within B and reachable from the beginning of B (called **upwards exposed uses**).
- KILL(B): Set of uses that, if they can be reached from the end of B, they cannot be reached from the beginning of B

2. Compute two other sets for propagation

- IN(B) is the set of uses that can be reached from the end of B
- OUT(B) is the set of uses that can be reached from the beginning of B

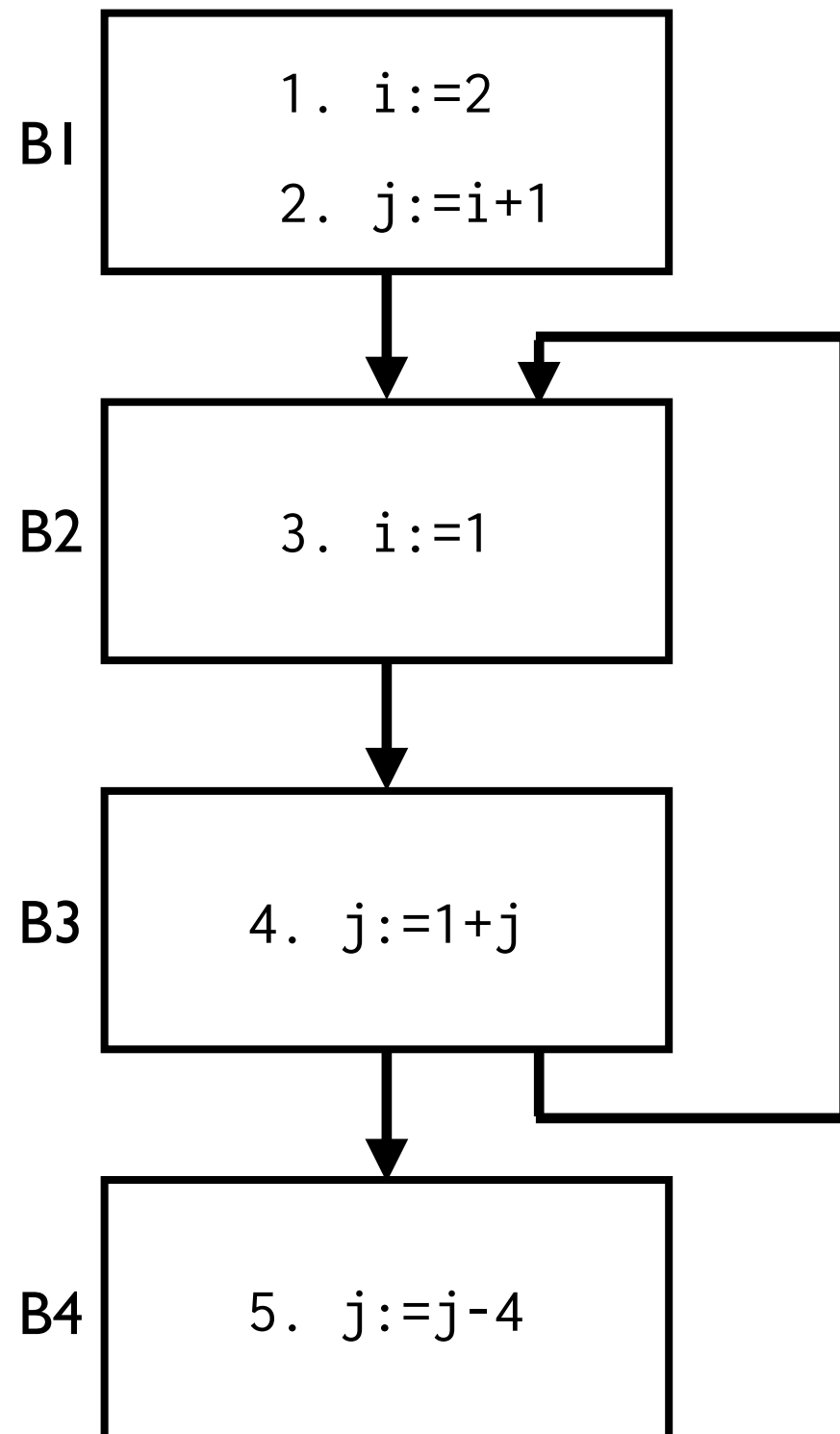
Iterative Dataflow Analysis



3. Propagation method:

- Initialise the $IN(B)$, $OUT(B)$ sets for all B
- Iterate (worklist) until there are no changes to the $IN(B)$, $OUT(B)$ sets
- Compute $IN(B)$, $OUT(B)$ as:
$$IN(B) = \bigcup OUT(S), S \in \text{succ}(B)$$
$$OUT(B) = \text{GEN}(B) \cup (IN(B) - \text{KILL}(B))$$

Iterative Dataflow Analysis



Similarities between RD and RU

- Local information (GEN and KILL) computed for each B
- IN and OUT sets defined: IN at point where data flows into B from outside B; OUT at point where data flow out of B
- Flow into block computed as union of predecessors in flow
- Iteration until no more changes

Differences between RD and RU

- RD flow is forward; RU flow is backward
- RD best ordering is depth-first (topological); RU best ordering is reverse depth-first (reverse topological)

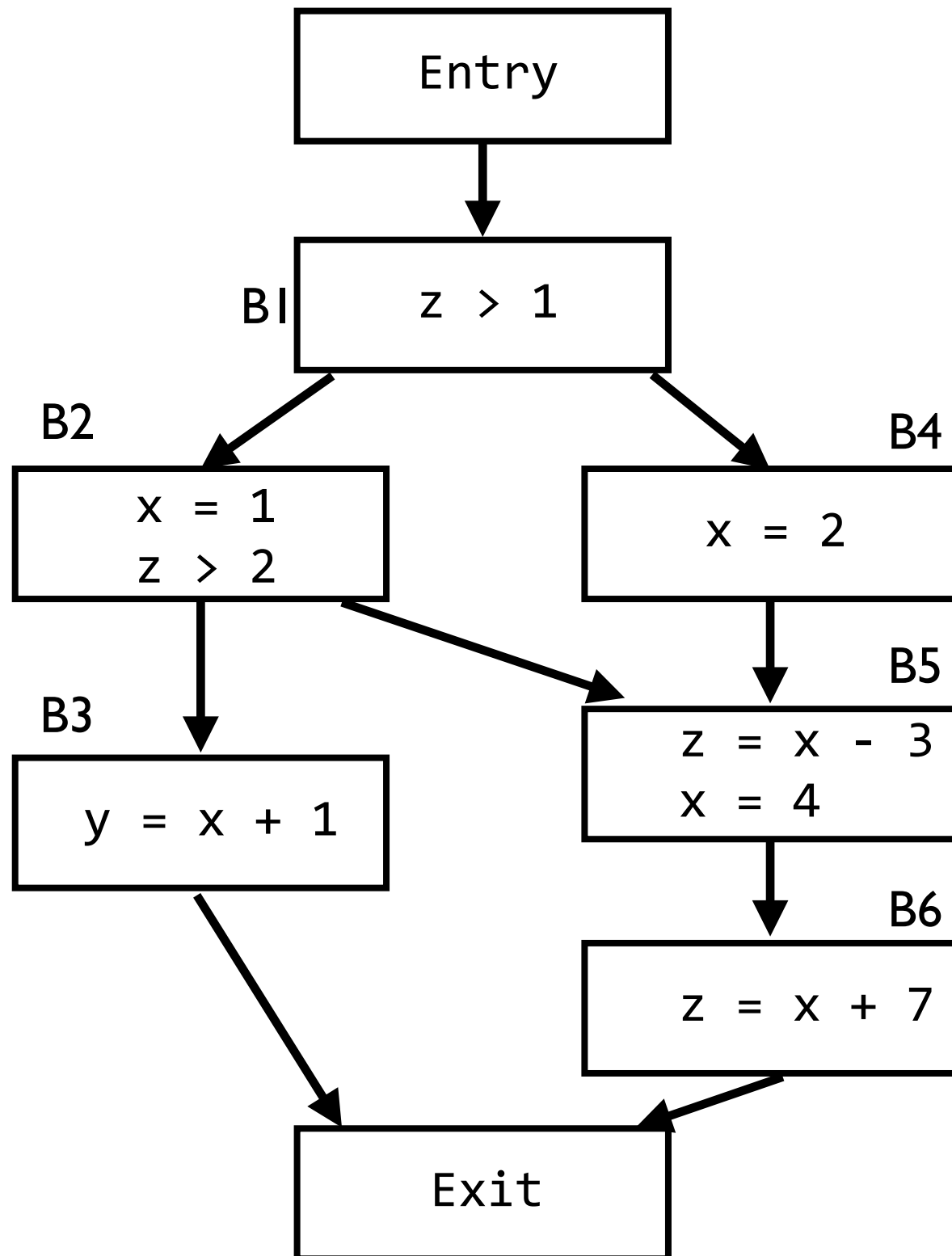
Outline

1. Reaching Definitions
2. Reachable Uses
3. DU Chains, UD Chains
4. Iterative Dataflow Analysis

DU-Chains, UD-Chains

- A **definition-use chain** (or **DU-chain**) for a definition D of variable v connects the D to all uses of v that it can reach
- A **use-definition chain** (or **UD-chain**) for a use U of variable v connects U to all definitions of v that reach it

DU-Chains



$\text{DU-chain}(x, B2): \{(x, B3), (x, B5)\}$

$\text{DU-chain}(x, B4): \{(x, B5)\}$

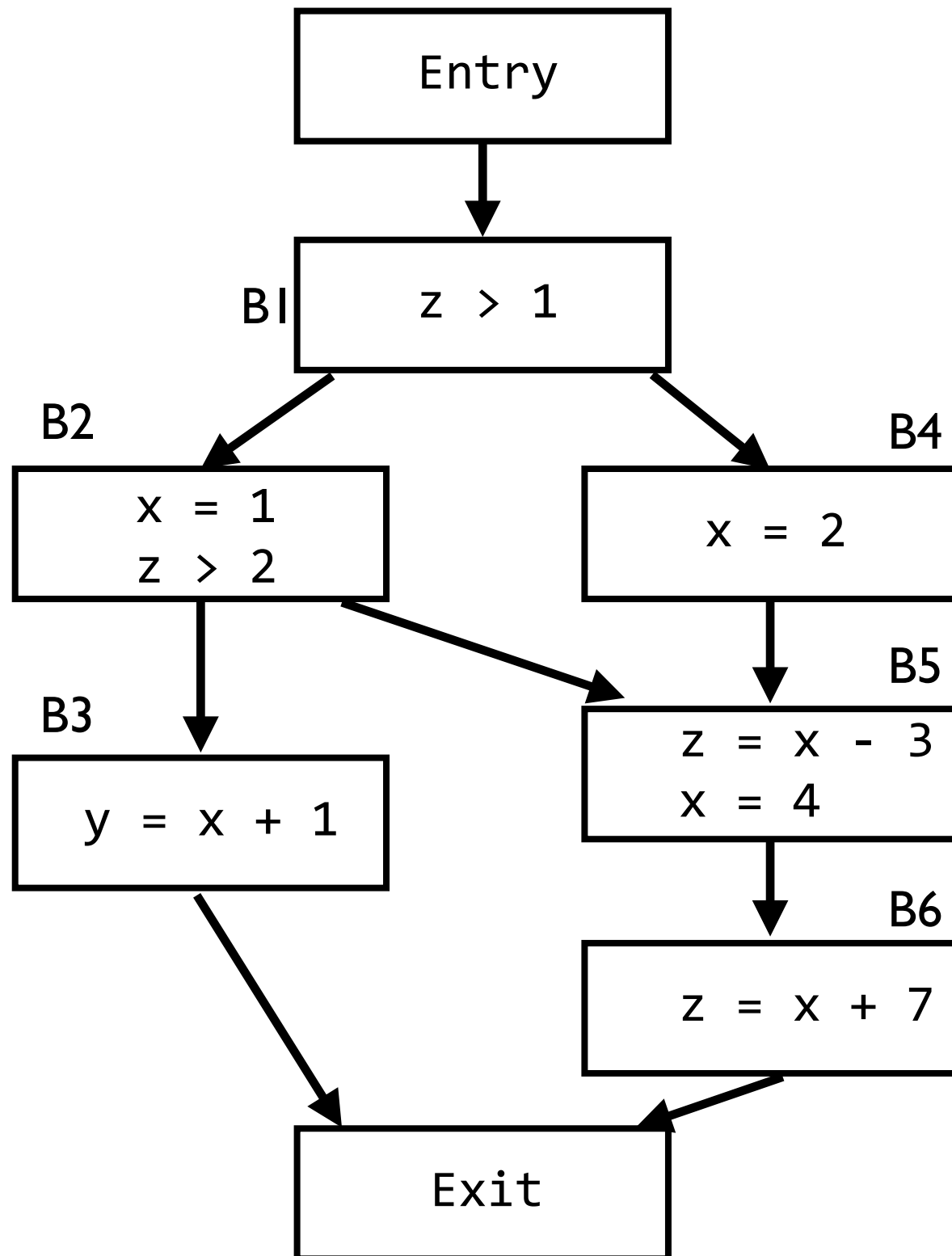
$\text{DU-chain}(x, B5): \{(x, B6)\}$

$\text{DU-chain}(y, B3): \{\}$

$\text{DU-chain}(z, B5): \{\}$

$\text{DU-chain}(z, B6): \{\}$

UD-Chains



UD-chain(z , B1): $\{\}$

UD-chain(z , B2): $\{\}$

UD-chain(x , B3): $\{(x, B2)\}$

UD-chain(x , B5): $\{(x, B2), (x, B4)\}$

UD-chain(x , B6): $\{(x, B5)\}$

Outline

1. Reaching Definitions
2. Reachable Uses
3. DU Chains, UD Chains
4. Iterative Dataflow Analysis

Space of Data Flow Analyses

	May	Must
Forward	??	??
Backward	??	??

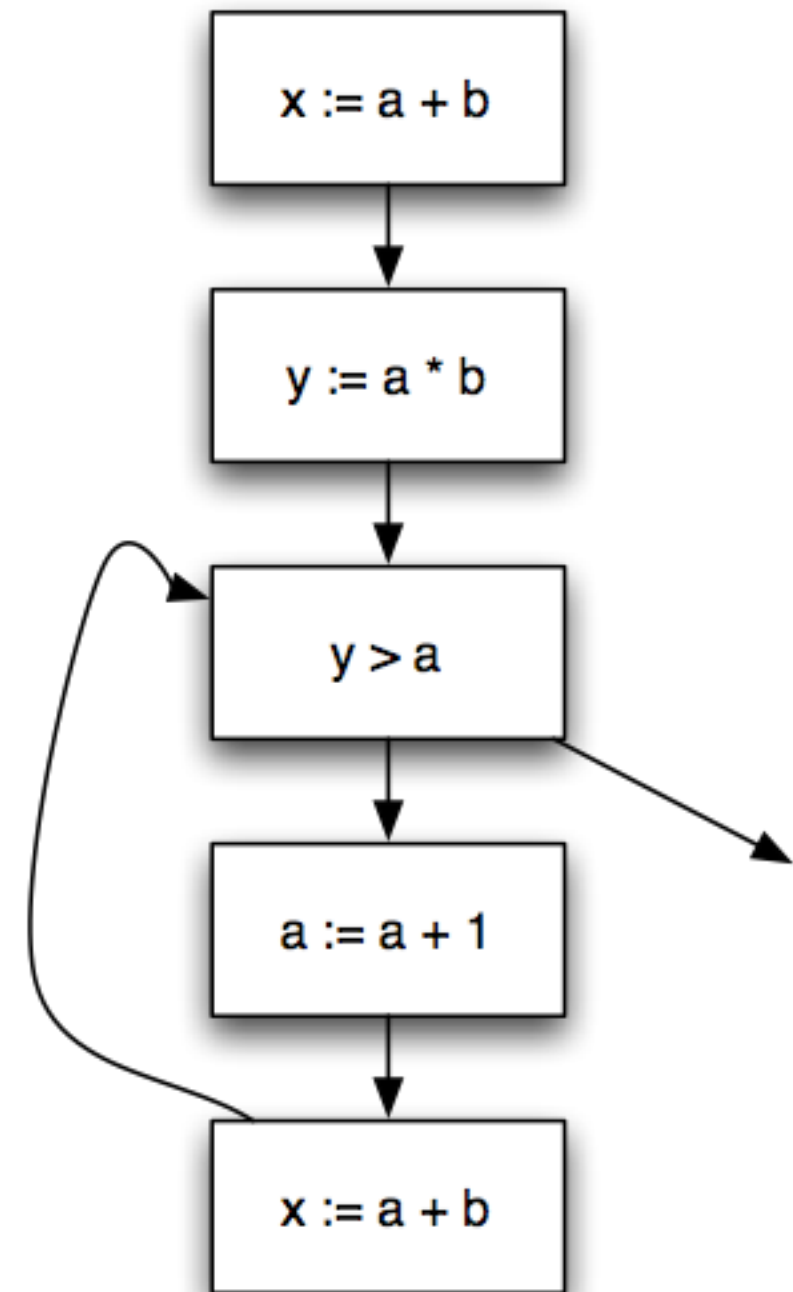
- Most data flow analyses can be classified this way
- Lots of literature on data flow analysis

Available Expressions

- An expression e is available at program point p if
 - e is computed on every path to p , and
 - the value of e has not changed since the last time e is computed on p
- Optimisation
 - If an expression is available, need not be recomputed
 - (At least, if it's still in a register somewhere)

Data Flow Facts

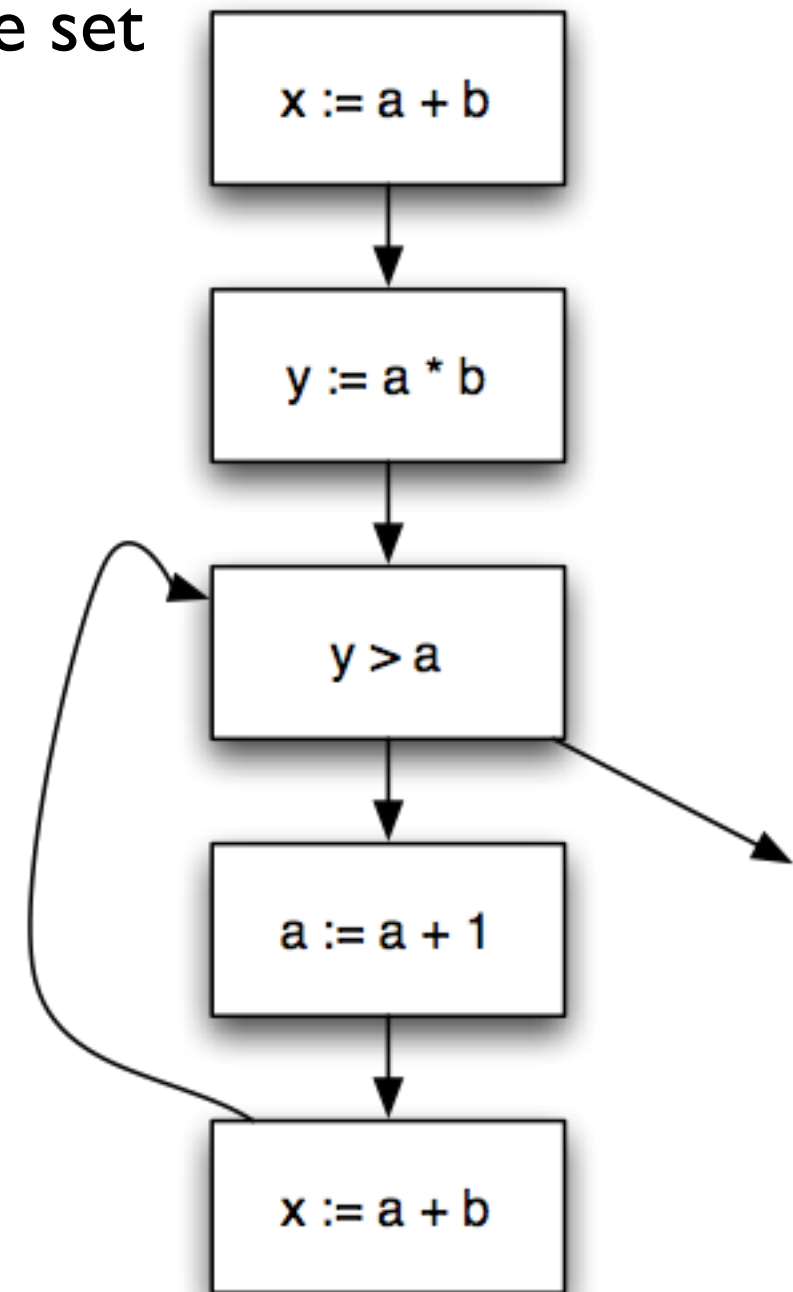
- Is expression e available?
- Facts:
 - $a + b$ is available
 - $a * b$ is available
 - $a + 1$ is available



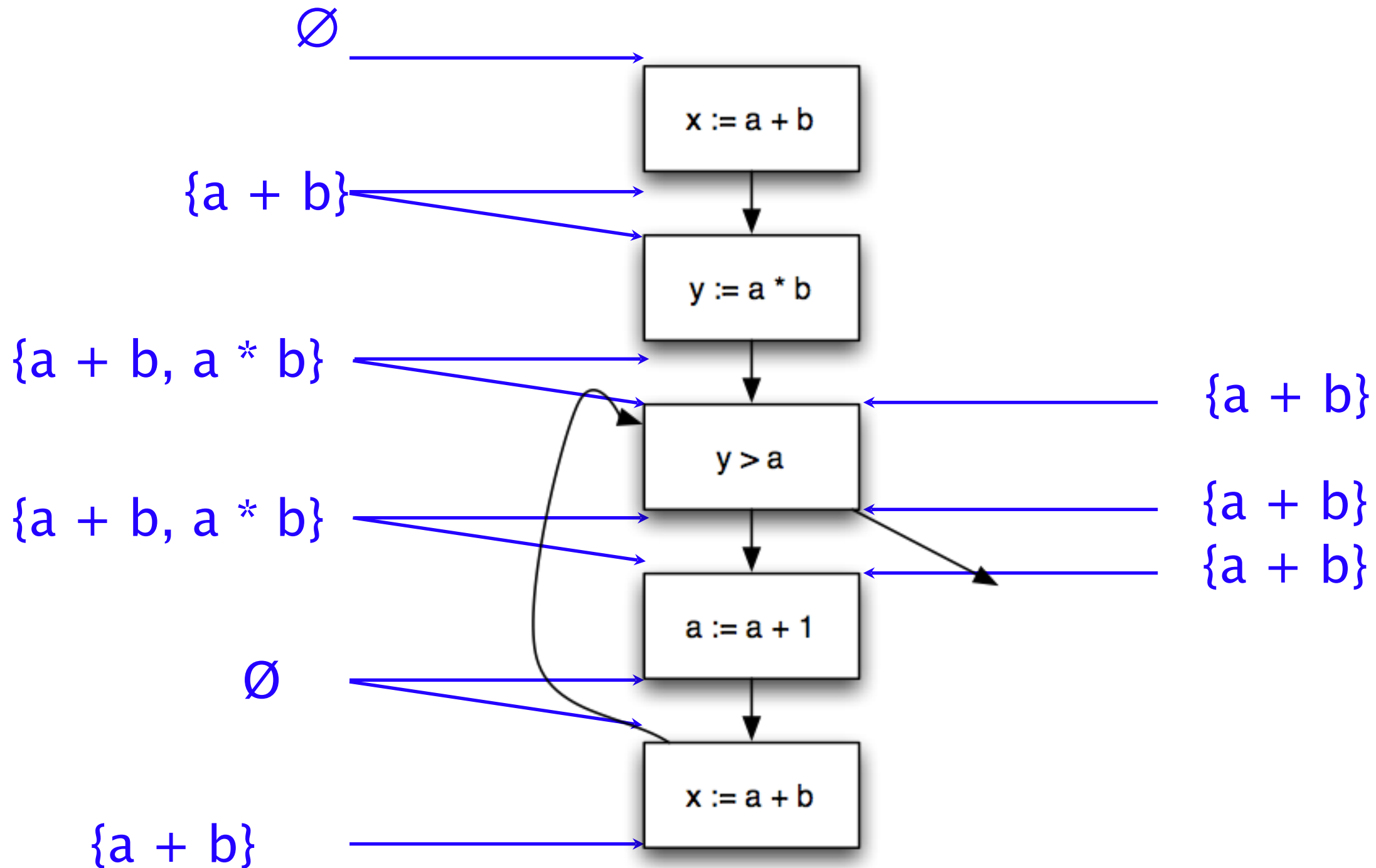
Gen and Kill

- What is the effect of each statement on the set of facts?

Stmt	Gen	Kill
$x := a + b$	$a + b$	
$y := a * b$	$a * b$	
$a := a + 1$		$a + 1,$ $a + b,$ $a * b$



Computing Available Expressions



Terminology

- Available expressions is a *forward must* problem
 - Forward = Data flow from **in** to **out**
 - Must = At join point, property must hold on all paths that are joined
- A *join point* is a program point where two branches meet

Data Flow Equations

- Let s be a statement
 - $\text{succ}(s) = \{ \text{immediate successor statements of } s \}$
 - $\text{pred}(s) = \{ \text{immediate predecessor statements of } s \}$
 - $\text{In}(s) = \text{program point just before executing } s$
 - $\text{Out}(s) = \text{program point just after executing } s$
- $\text{In}(s) = \bigcap_{s' \in \text{pred}(s)} \text{Out}(s')$
- $\text{Out}(s) = \text{Gen}(s) \cup (\text{In}(s) - \text{Kill}(s))$

Space of Data Flow Analyses

	May	Must
Forward	??	Available expressions
Backward	??	??

- Most data flow analyses can be classified this way
- Lots of literature on data flow analysis

Liveness Analysis

- A variable v is *live* at program point p if
 - v will be used on some execution path originating from p ...
 - before v is overwritten
- Optimisation
 - If a variable is not live, no need to keep it in a register
 - If variable is dead at assignment, can eliminate assignment
- Uninitialised variable detection
 - No variable should be live at program entry point

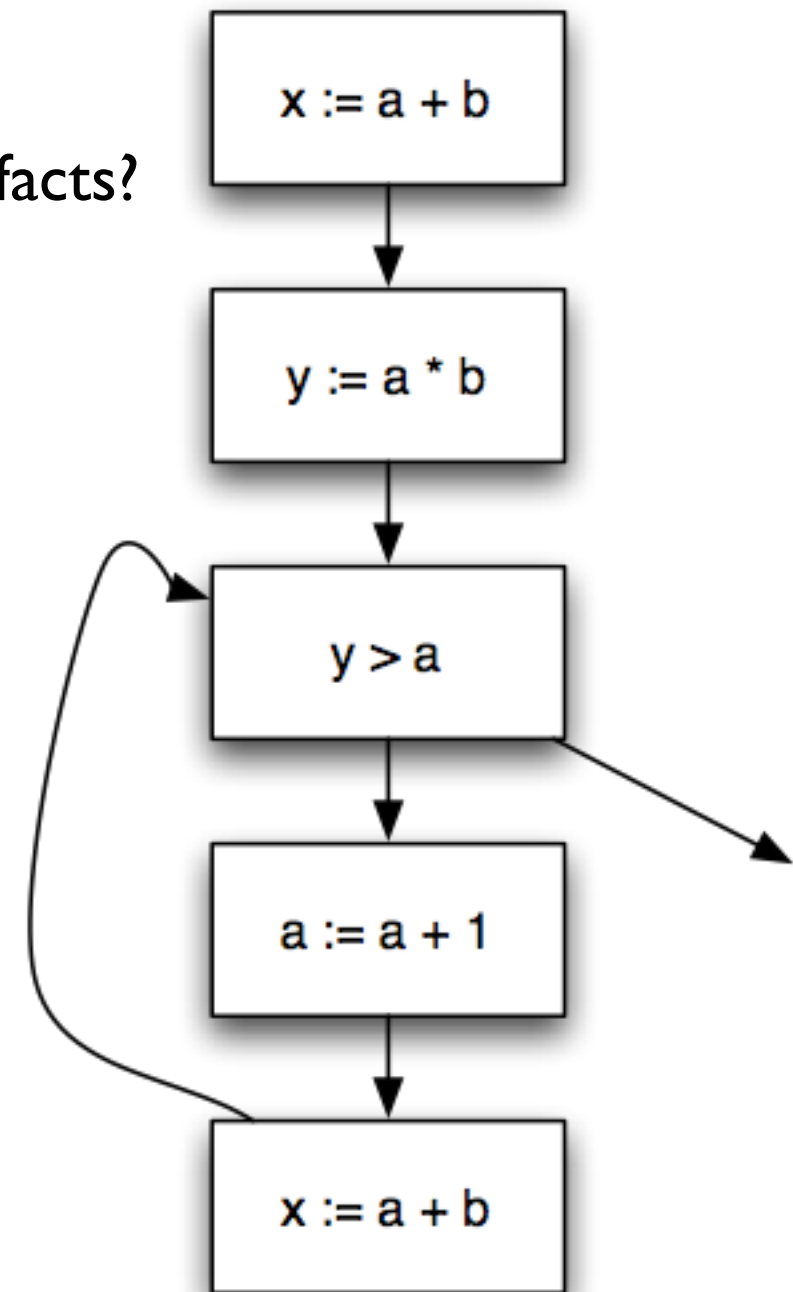
Data Flow Equations

- $\text{Out}(s) = \bigcup \text{In}(s'), \text{ for } s' \in \text{Succ}(s)$
- $\text{In}(s) = \text{Gen}(s) \cup (\text{Out}(s) - \text{Kill}(s))$
- $\text{Gen}(n) = \{ v \mid v \text{ is used at } n \}$
- $\text{Kill}(n) = \{ v \mid v \text{ is modified at } n \}$

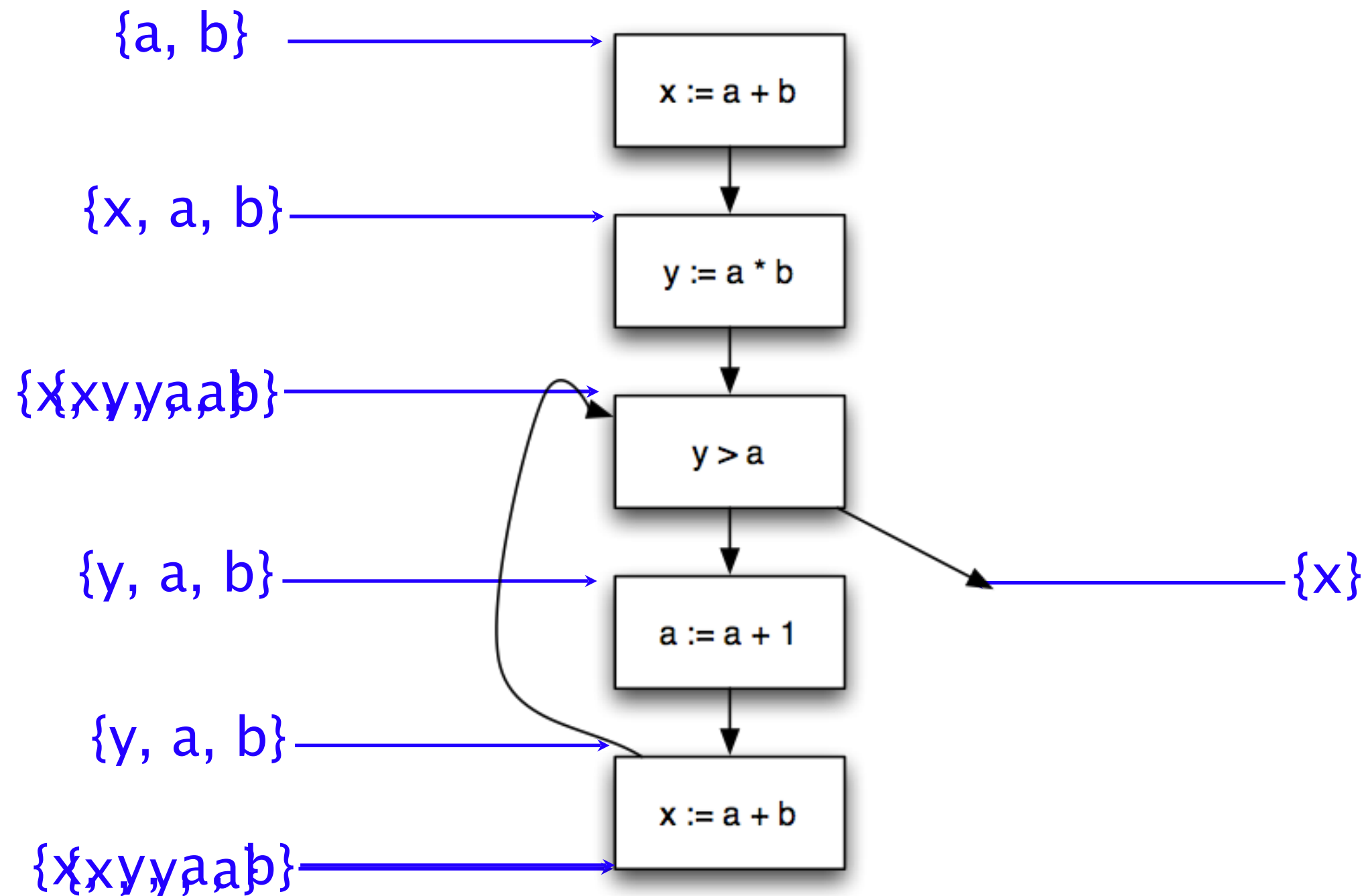
Gen and Kill

- What is the effect of each statement on the set of facts?

Stmt	Gen	Kill
$x := a + b$	a, b	x
$y := a * b$	a, b	y
$y > a$	a, y	
$a := a + 1$	a	a



Computing Live Variables



Liveness Analysis

- Liveness is a **backward may** problem
 - To know if variable live, need to look at **future uses**
 - Variable is live if used on **some path**
- In contrast, available expressions is a **forward must** analysis
 - Data flow propagates in same direction as CFG edges
 - Expression is available only if available on **all paths**

Space of Data Flow Analyses

	May	Must
Forward	??	Available expressions
Backward	Live variables	??

- Most data flow analyses can be classified this way
- Lots of literature on data flow analysis

Very Busy Expressions

- An expression e is **very busy** at point p if
 - On **every path** from p , expression e is evaluated before the value of e is changed
- Optimization: Can hoist very busy expression computation
- Very Busy Expression is a **backward must** problem

Space of Data Flow Analyses

	May	Must
Forward	??	Available expressions
Backward	Live variables	Very busy expressions

- Most data flow analyses can be classified this way
- Lots of literature on data flow analysis

Space of Data Flow Analyses

	May	Must
Forward	Reaching definitions	Available expressions
Backward	Live variables Reachable uses	Very busy expressions

- Most data flow analyses can be classified this way
- Lots of literature on data flow analysis

Iterative Solution of Dataflow Equations

- Initialize values with a **first estimate** of the answer:
 - “Any path” problems start with “nothing” (empty set) at each node
 - “All paths” problems start with “everything” (set of all possible values = union of all “gen” sets)
- Repeat until nothing changes
 - Pick some node and recalculate (new estimate)

This will converge on a “fixed point” solution where every new calculation produces the same value as the previous guess.